

Attitude Dynamics and Control of a Nano-Satellite Orbiting Mars

Gunnar Montseny Gens*

University of Colorado Boulder, Boulder, Colorado 80303

This work studies the attitude dynamics and control of a nano-satellite in low Mars orbit (LMO) subject to multiple pointing requirements, including nadir, Sun, and communication with mother spacecraft in a geosynchronous Mars orbit (GMO). The reference attitudes for these requirements are constructed from orbital geometry, assuming circular two-body motion. The spacecraft is modeled as a rigid body and controlled through a proportional-derivative (PD) law to drive the spacecraft toward different reference attitudes. The spacecraft attitude motion is numerically integrated through a fixed-step 4th order Runge–Kutta routine, leading to a full mission scenario simulation. Simulation results validate the modeling and control framework across all mission scenarios.

I. Introduction

SPACECRAFT operating in planetary orbits are often required to satisfy multiple pointing constraints that vary throughout the mission. In particular, observation, communication, and power generation objectives impose distinct attitude requirements that cannot be achieved simultaneously, requiring the spacecraft to transition between different pointing modes over the course of an orbit.

In this work, we consider a nano-satellite in a circular low Mars orbit (LMO) performing scientific observations. The collected data must then be transmitted to the mother (relay) spacecraft in a circular geosynchronous Mars orbit (GMO). Further, to keep the batteries from draining all the way, periodically the satellite must point its solar panels at the Sun to recharge. Thus, three mission goals must be considered by the nano-satellite:

- 1) Point science instruments toward the Mars nadir direction.
- 2) Point communication hardware at the mother satellite.
- 3) Point the solar arrays at the Sun.

The high-level goal of this paper is to construct these reference frames and design an attitude control law to achieve these pointing objectives. In order to satisfy the mission requirements, we drive the spacecraft's body frame towards various reference frames that correspond to desired attitudes. Note that the spacecraft is modeled as a rigid body with perfect state knowledge and no external disturbances.

II. Mission Overview

This section presents all the details of the mission to the reader. The details of the different mission pointing scenarios are specified, along with the frames used in this work. In addition, we provide more information about the initial state of the nano-satellite.

A. Frame Definition

Let $\mathcal{N} = \{\hat{n}_1, \hat{n}_2, \hat{n}_3\}$ be the inertial frame with its origin set at the center of mass of Mars, with the corresponding unit vectors shown in Figure 1. Also consider body-frame $\mathcal{B} = \{\hat{b}_1, \hat{b}_2, \hat{b}_3\}$, with origin at the center of mass of the LMO spacecraft and depicted in Figure 2. The LMO spacecraft is equipped with an antenna to communicate with the mothercraft (aligned with $-\hat{b}_1$), a sensor to observe Mars (aligned with $\hat{b}_1$), and solar panels for power (panel normal is along $\hat{b}_3$).

Further, a general circular orbit frame $\mathcal{O} = \{\hat{i}_r, \hat{i}_\theta, \hat{i}_h\}$ is used for initial derivations. $\hat{i}_r$ points to the spacecraft, $\hat{i}_h$ is the direction of the angular momentum vector, and $\hat{i}_\theta = \hat{i}_h \times \hat{i}_r$. However, a Hill frame $\mathcal{H} = \{\hat{h}_r, \hat{h}_\theta, \hat{h}_h\}$ is used throughout the paper to specifically describe a circular LMO. Note that the Hill frame is given through the (3-1-3) Euler angle set (Ω, i, θ) . See Figure 1 for an illustration of the Hill frame with respect to the inertial frame.

*Ann and H.J. Smead Department of Aerospace Engineering Sciences, University of Colorado Boulder, 3775 Discovery Drive, Boulder, CO 80303, USA

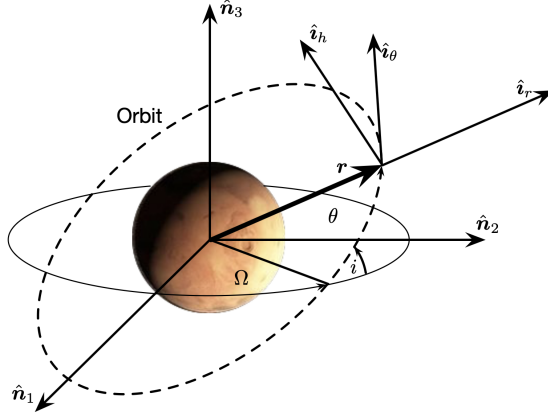


Fig. 1 Illustration of Inertial Frame $\mathcal{N}$ and Hill frame $\mathcal{H} = \{\hat{i}_r, \hat{i}_\theta, \hat{i}_h\}$.

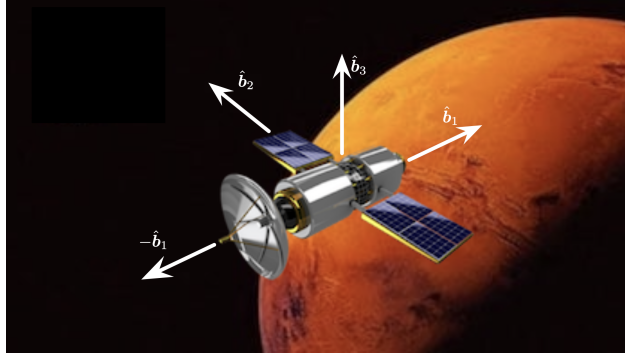


Fig. 2 Illustration of the spacecraft component directions, and definition of the body-frame $\mathcal{B}$.

In addition, three body reference frames are considered and constructed in Section III: Sun-pointing reference frame $\mathcal{R}_s$, nadir-pointing reference frame $\mathcal{R}_n$, and GMO-pointing reference frame $\mathcal{R}_c$.

B. Mission Orbit Overview

As mentioned, in this study we assume that the orbits of both spacecrafts are circular. Consequently, the orbits have a constant radius and a constant angular rate. Let the Mars gravity constant be $\mu_{Mars} = 42828.3 \text{ km}^3/\text{s}^2$. Then, the constant rate of a circular orbit with radius r can be calculated with the following expression:

$$\dot{\theta}(r) = \sqrt{\frac{\mu_{Mars}}{r^3}}. \quad (1)$$

Table 1 presents the radius, angular rates, and initial (3-1-3) Euler angles for both orbits.

Table 1 Orbit Overview Details

Spacecraft	r (km)	$\dot{\theta}$ (rad/s)	$\Omega(t_0)$ (deg)	$i(t_0)$ (deg)	$\theta(t_0)$ (deg)
LMO	3796.19	0.000884797	20	30	60
GMO	20424.2	0.0000709003	0	0	250

C. Mission Pointing Scenarios

This section defines the various attitude pointing modes for the spacecraft. For simplification, it can be assumed that the sun is an infinite distance away and always in the $\hat{n}_2$ direction as illustrated in Figure 3. During the mission, the control law must detumble the satellite and either point its antenna ($-\hat{b}_1$) toward the GMO mother satellite, its sensor ($\hat{b}_1$) at Mars (nadir direction), or its solar panels ($\hat{b}_3$) at the Sun. Due to the high power demands of the telescope, the spacecraft must point its solar panels toward the Sun whenever it is on the sunlit side of Mars (i.e., positive satellite $\hat{n}_2$ position coordinate). In other words, when pointing at the Sun, the spacecraft must align its solar panel axis $\hat{b}_3$ with the $\hat{n}_2$ direction.

When the spacecraft is on the shaded side of Mars (i.e., over the hemisphere with negative $\hat{n}_2$ position coordinate), it must enter either communication or science mode. In science mode, the science platform axis $\hat{b}_1$ must point toward the center of Mars, i.e., in the nadir direction. The communication mode requires that the angular separation between the LMO and GMO position vectors be less than 35° . In this mode, the nano-satellite communication platform axis $-\hat{b}_1$ must point toward the GMO mother spacecraft. Table 2 summarizes these pointing requirements.

Table 2 Spacecraft Pointing Scenario Summary

Orbital Situation	Primary Pointing Scenario Goals
Spacecraft on sunlit Mars side	Point solar axis $\hat{b}_3$ at Sun
Spacecraft not on sunlit Mars side and GMO visible	Point antenna axis $-\hat{b}_1$ at mothercraft
Spacecraft not on sunlit Mars side and GMO not visible	Point sensor axis $\hat{b}_1$ along Mars nadir direction

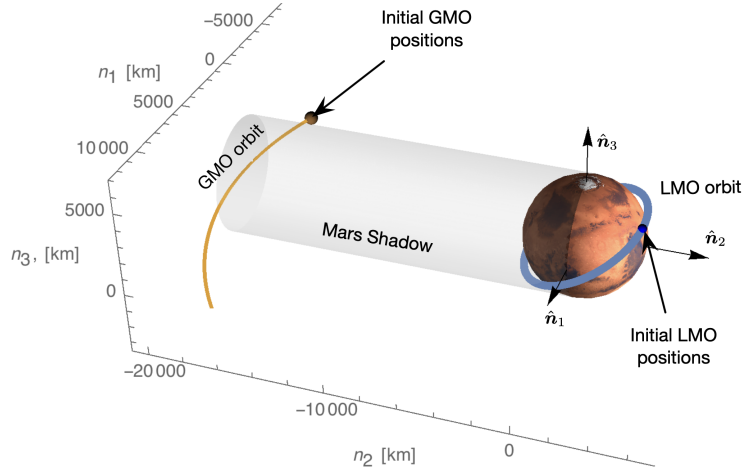


Fig. 3 Mission Orbit Scenario Illustration.

D. Spacecraft Attitude States

The spacecraft's initial attitude is given through the following Modified Rodrigues Parameter (MRP) set:

$$\vec{\sigma}_{B/N}(t_0) = \begin{bmatrix} 0.3 \\ -0.4 \\ 0.5 \end{bmatrix}. \quad (2)$$

The initial body angular velocity with respect to the inertial frame is given by:

$${}^{\mathcal{B}}\vec{\omega}_{\mathcal{B}/\mathcal{N}}(t_0) = \begin{bmatrix} 1.00 \\ 1.75 \\ -2.20 \end{bmatrix} \text{ deg/s.} \quad (3)$$

The rigid LMO spacecraft inertia tensor is given by:

$${}^{\mathcal{B}}[I] = \begin{bmatrix} 10 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 7.5 \end{bmatrix} \text{ kg m}^2. \quad (4)$$

Note that ${}^{\mathcal{B}}[I]$ is a diagonal matrix, indicating that the three axes in body frame $\mathcal{B}$ are principal axes of our rigid LMO spacecraft.

III. Modeling and Control Framework

A. Orbit Simulation

The position vector on a circular orbit of radius r can be described through frame $\mathcal{O}$ as ${}^{\mathcal{O}}\vec{r} = r\hat{r}$. To obtain the inertial velocity, we apply the Transport Theorem:

$$\dot{\vec{r}} = \frac{{}^{\mathcal{N}}d}{{}^{\mathcal{N}}dt} ({}^{\mathcal{O}}\vec{r}) = \frac{{}^{\mathcal{O}}d}{{}^{\mathcal{O}}dt} ({}^{\mathcal{O}}\vec{r}) + {}^{\mathcal{O}}\vec{\omega}_{\mathcal{O}/\mathcal{N}} \times {}^{\mathcal{O}}\vec{r} = r\dot{\hat{r}} + {}^{\mathcal{O}}\vec{\omega}_{\mathcal{O}/\mathcal{N}} \times {}^{\mathcal{O}}\vec{r} \quad (5)$$

By definition of a circular orbit, we know that ${}^{\mathcal{O}}\vec{\omega}_{\mathcal{O}/\mathcal{N}} = \dot{\theta}\hat{h}$. In addition, since the $\mathcal{O}$ frame follows a circular orbit, $\dot{r} = 0$, and the inertial velocity can be written as

$${}^{\mathcal{O}}\dot{\vec{r}} = r\dot{\theta}\hat{t}_{\theta} = \begin{bmatrix} 0 \\ r\dot{\theta} \\ 0 \end{bmatrix}. \quad (6)$$

To express the vectors ${}^{\mathcal{O}}\vec{r}$ and ${}^{\mathcal{O}}\dot{\vec{r}}$ in the $\mathcal{N}$ frame, we left-multiply them by the corresponding Direction Cosine Matrix (DCM):

$${}^{\mathcal{N}}\vec{r}(t) = [{}^{\mathcal{N}}\mathcal{O}] \Big|_t {}^{\mathcal{O}}\vec{r}(t), \quad {}^{\mathcal{N}}\dot{\vec{r}}(t) = [{}^{\mathcal{N}}\mathcal{O}] \Big|_t {}^{\mathcal{O}}\dot{\vec{r}}(t), \quad (7)$$

where $[{}^{\mathcal{N}}\mathcal{O}]$ is the DCM that maps the $\mathcal{N}$ frame to the $\mathcal{O}$ frame. It can be defined with the 3-1-3 Euler angles $(\Omega, i, \theta(t))$:

$$[{}^{\mathcal{N}}\mathcal{O}] \Big|_t = \begin{bmatrix} \cos \theta(t) \cos \Omega - \sin \theta(t) \cos i \sin \Omega & \cos \theta(t) \sin \Omega + \sin \theta(t) \cos i \cos \Omega & \sin \theta(t) \sin i \\ -\sin \theta(t) \cos \Omega - \cos \theta(t) \cos i \sin \Omega & -\sin \theta(t) \sin \Omega + \cos \theta(t) \cos i \cos \Omega & \cos \theta(t) \sin i \\ \sin i \sin \Omega & -\sin i \cos \Omega & \cos i \end{bmatrix}. \quad (8)$$

Note that we assume that both spacecraft are in a circular orbit, meaning that the true anomaly $\theta(t)$ can be expressed as:

$$\theta(t) = \theta(t_0) + \dot{\theta}(t - t_0), \quad (9)$$

where $\dot{\theta}$ is constant, and $\theta(t_0)$ is the true anomaly of the chosen orbit at time t_0 . These values are provided for both the LMO and GMO case in Table 1.

B. Orbit Frame Orientation

Consider the Hill frame $\mathcal{H} = \{\hat{i}_r, \hat{i}_\theta, \hat{i}_h\}$, the orbit frame of the LMO satellite. We define these vectors as

$$\hat{i}_r = \frac{\vec{r}_{\text{LMO}}}{\|\vec{r}_{\text{LMO}}\|}, \quad \hat{i}_\theta = \hat{i}_h \times \hat{i}_r, \quad \hat{i}_h = \frac{\vec{r}_{\text{LMO}} \times \dot{\vec{r}}_{\text{LMO}}}{\|\vec{r}_{\text{LMO}} \times \dot{\vec{r}}_{\text{LMO}}\|},$$

where r_{LMO} can be found in Table 1, and $\dot{r}_{\text{LMO}} = r_{\text{LMO}} \dot{\theta}_{\text{LMO}}$. To calculate the DCM $[\mathcal{H}N]_t$, we use Eq. (8) with $(\Omega, i, \theta) = (\Omega_{\text{LMO}}, i_{\text{LMO}}, \theta_{\text{LMO}}(t))$, where Ω_{LMO} and i_{LMO} can be found in Table 1, and $\theta_{\text{LMO}}(t)$ can be computed through Eq. (9).

C. Sun-Pointing Frame Orientation

To point the spacecraft solar panels axis $\hat{b}_3$ at the sun, the reference frame $\mathcal{R}_s = \{\hat{r}_1, \hat{r}_2, \hat{r}_3\}$ must be chosen such that $\hat{r}_3$ axis points in the sun direction ($\hat{n}_2$ in this scenario). Further, we assume that the first axis $\hat{r}_1$ points in the $-\hat{n}_1$ direction. Consequently, we have that $\hat{r}_1 = -\hat{n}_1$, and $\hat{r}_3 = \hat{n}_2$. To ensure that $\mathcal{R}_s$ is a right-handed frame, we find the last unit vector of the frame through $\hat{r}_2 = \hat{r}_3 \times \hat{r}_1 = \hat{n}_3$.

By definition, the DCM that maps vectors from the inertial frame $\mathcal{N}$ to the Sun-pointing frame $\mathcal{R}_s$ is:

$$[\mathcal{R}_s N] = \begin{bmatrix} {}^N \hat{r}_1^T \\ {}^N \hat{r}_2^T \\ {}^N \hat{r}_3^T \end{bmatrix} = \begin{bmatrix} -{}^N \hat{n}_1^T \\ {}^N \hat{n}_3^T \\ {}^N \hat{n}_2^T \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}. \quad (10)$$

Note that ${}^N \vec{\omega}_{\mathcal{R}_s/N} = \vec{0}$ because the reference frame $\mathcal{R}_s$ is constant in time with respect to the inertial frame $\mathcal{N}$.

D. Nadir-Pointing Reference Frame Orientation

To point the spacecraft sensor platform axis $\hat{b}_1$ towards the center of Mars or nadir direction, the reference frame $\mathcal{R}_n = \{\hat{r}_1, \hat{r}_2, \hat{r}_3\}$ must be chosen such that $\hat{r}_1$ axis points towards the planet ($\hat{r}_1 = -\hat{i}_r$). Further, we assume that the second axis $\hat{r}_2$ points in the velocity direction $\hat{i}_\theta$. To ensure that $\mathcal{R}_n$ is a right-handed frame, we find the last unit vector of the frame through $\hat{r}_3 = \hat{r}_1 \times \hat{r}_2 = -\hat{i}_h$. The DCM $[\mathcal{R}_n \mathcal{H}]$ can be calculated similarly to last section:

$$[\mathcal{R}_n \mathcal{H}] = \begin{bmatrix} {}^{\mathcal{H}} \hat{r}_1^T \\ {}^{\mathcal{H}} \hat{r}_2^T \\ {}^{\mathcal{H}} \hat{r}_3^T \end{bmatrix} = \begin{bmatrix} -{}^{\mathcal{H}} \hat{i}_r^T \\ {}^{\mathcal{H}} \hat{i}_\theta^T \\ -{}^{\mathcal{H}} \hat{i}_h^T \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & -1 \end{bmatrix}. \quad (11)$$

Through properties of DCMs, we know that:

$$[R_n N] = [R_n \mathcal{H}] [\mathcal{H} N], \quad (12)$$

The angular velocity of the reference frame $\mathcal{R}_n$ with respect to the inertial frame can be calculated through the following expression:

$${}^{\mathcal{H}} \vec{\omega}_{\mathcal{R}_n/N} = {}^{\mathcal{H}} \vec{\omega}_{\mathcal{R}_n/\mathcal{H}} + {}^{\mathcal{H}} \vec{\omega}_{\mathcal{H}/N} = \dot{\theta}_{\text{LMO}} \hat{i}_h. \quad (13)$$

Lastly, we multiply the angular velocity vector by $[N \mathcal{H}]$ to obtain the angular velocity expressed with the inertial frame:

$${}^N \vec{\omega}_{\mathcal{R}_n/N} = [N \mathcal{H}] \begin{bmatrix} 0 \\ 0 \\ \dot{\theta}_{\text{LMO}} \end{bmatrix}. \quad (14)$$

It is important to note that both $[\mathcal{R}_n \mathcal{H}]$ and $\|{}^N \vec{\omega}_{\mathcal{R}_n/N}\|$ are constant in time.

E. GMO-Pointing Reference Frame Orientation

To point the nano-satellite communication platform axis $-\hat{b}_1$ towards the GMO mother spacecraft, the communication reference frame $\mathcal{R}_c = \{\hat{r}_1, \hat{r}_2, \hat{r}_3\}$ must be chosen such that $-\hat{r}_1$ axis points towards the GMO satellite location. We define a relative position vector $\Delta\vec{r}(t) = \vec{r}_{\text{GMO}}(t) - \vec{r}_{\text{LMO}}(t)$, such that $\hat{r}_1 = -\Delta\vec{r}/|\Delta\vec{r}|$. Further, to fully define a three-dimensional reference frame, assume the second axis is defined as

$$\hat{r}_2 = \frac{\Delta\vec{r} \times \hat{n}_3}{|\Delta\vec{r} \times \hat{n}_3|} \quad (15)$$

while the third is then defined as $\hat{r}_3 = \hat{r}_1 \times \hat{r}_2$. The DCM $[\mathcal{R}_c\mathcal{N}]$ can be calculated with the following expression:

$$[\mathcal{R}_c\mathcal{N}] = \begin{bmatrix} N\hat{r}_1^T \\ N\hat{r}_2^T \\ N\hat{r}_3^T \end{bmatrix}, \quad (16)$$

which is not constant in time. As a result, to calculate the angular velocity of this reference frame with respect to the inertial frame, we consider the differential equation for $[\mathcal{R}_c\mathcal{N}]$:

$$\mathcal{R}_c [\tilde{\omega}_{\mathcal{R}_c/\mathcal{N}}] \Big|_t = -\frac{d}{dt} ([\mathcal{R}_c\mathcal{N}]) \Big|_t [\mathcal{R}_c\mathcal{N}]^T \Big|_t, \quad (17)$$

where $[\tilde{\omega}]$ is the skew-operator of vector $\vec{\omega}$. We numerically calculate the derivative of $[\mathcal{R}_c\mathcal{N}]$ through central differences:

$$\frac{d}{dt} ([\mathcal{R}_c\mathcal{N}]) \Big|_t \approx \frac{[\mathcal{R}_c\mathcal{N}] \Big|_{t+\Delta t} - [\mathcal{R}_c\mathcal{N}] \Big|_{t-\Delta t}}{2\Delta t} \quad (18)$$

Lastly, we extract the necessary components from the skew-operator:

$${}^{\mathcal{N}}\vec{\omega}_{\mathcal{R}_c/\mathcal{N}}(t) = [\mathcal{R}_c\mathcal{N}]^T \Big|_t \begin{bmatrix} \mathcal{R}_c [\tilde{\omega}_{\mathcal{R}_c/\mathcal{N}}]_{32} \Big|_t \\ \mathcal{R}_c [\tilde{\omega}_{\mathcal{R}_c/\mathcal{N}}]_{13} \Big|_t \\ \mathcal{R}_c [\tilde{\omega}_{\mathcal{R}_c/\mathcal{N}}]_{21} \Big|_t \end{bmatrix}. \quad (19)$$

F. Attitude Error Evaluation

For our attitude feedback control algorithm we need to compute the attitude and angular velocity tracking errors of current body frame $\mathcal{B}$ relative to a reference frame $\mathcal{R}$ of interest. For our mission, $\mathcal{R} \in \{\mathcal{R}_s, \mathcal{R}_n, \mathcal{R}_c\}$. We assume that the known variables are the attitude states $\sigma_{\mathcal{B}/\mathcal{N}}(t)$, ${}^{\mathcal{B}}\vec{\omega}_{\mathcal{B}/\mathcal{N}}(t)$, as well as the current reference frame DCM $[\mathcal{R}\mathcal{N}] \Big|_t$ and current reference frame rate ${}^{\mathcal{N}}\vec{\omega}_{\mathcal{R}/\mathcal{N}}(t)$. With these variables, we develop a formulation to compute the associated tracking errors $\sigma_{\mathcal{B}/\mathcal{R}}(t)$, ${}^{\mathcal{B}}\vec{\omega}_{\mathcal{B}/\mathcal{R}}(t)$. The first step is to calculate the DCM between the inertial frame and the current body frame through the following expression:

$$[\mathcal{B}\mathcal{N}] \Big|_t = [I_{3 \times 3}] + \frac{8[\tilde{\sigma}_{\mathcal{B}/\mathcal{N}}]^2 - 4(1 - \sigma_{\mathcal{B}/\mathcal{N}}^2) [\tilde{\sigma}_{\mathcal{B}/\mathcal{N}}]}{(1 + \sigma_{\mathcal{B}/\mathcal{N}}^2)^2} \quad (20)$$

With this DCM, we can now calculate the angular velocity tracking error:

$${}^{\mathcal{B}}\vec{\omega}_{\mathcal{B}/\mathcal{R}}(t) = {}^{\mathcal{B}}\vec{\omega}_{\mathcal{B}/\mathcal{N}}(t) + {}^{\mathcal{B}}\vec{\omega}_{\mathcal{N}/\mathcal{R}}(t) = {}^{\mathcal{B}}\vec{\omega}_{\mathcal{B}/\mathcal{N}}(t) - [\mathcal{B}\mathcal{N}] \Big|_t {}^{\mathcal{N}}\vec{\omega}_{\mathcal{R}/\mathcal{N}}(t), \quad (21)$$

To calculate the associated attitude tracking error, we have to compute the DCM between the reference frame $\mathcal{R}$ and body frame $\mathcal{B}$:

$$[\mathcal{B}\mathcal{R}] \Big|_t = [\mathcal{B}\mathcal{N}] \Big|_t [\mathcal{R}\mathcal{N}]^T \Big|_t. \quad (22)$$

Lastly, we calculate the attitude tracking error through C2MRP, a function that transforms DCM into MRPs (explanation of the function is provided below):

$$\vec{\sigma}_{\mathcal{B}/\mathcal{R}}(t) = \text{C2MRP} \left([\mathcal{B}\mathcal{R}] \Big|_t \right). \quad (23)$$

C2MRP

Consider an MRP representation $\vec{\sigma} \in \mathbb{R}^3$ and its corresponding DCM, $[C]$. To extract the MRP set from the DCM, we first compute the corresponding Euler Parameters (EPs) representation $\vec{\beta} = (\beta_0, \beta_1, \beta_2, \beta_3)$ using Sheppard's method.

Step 1: Find the largest value among

$$\begin{aligned} \beta_0^2 &= \frac{1}{4} (1 + \text{tr}([C])), & \beta_1^2 &= \frac{1}{4} (1 + 2C_{11} - \text{tr}([C])), \\ \beta_2^2 &= \frac{1}{4} (1 + 2C_{22} - \text{tr}([C])), & \beta_3^2 &= \frac{1}{4} (1 + 2C_{33} - \text{tr}([C])). \end{aligned}$$

Step 2: Compute the remaining EPs using

$$\begin{aligned} \beta_0\beta_1 &= \frac{C_{23} - C_{32}}{4}, & \beta_0\beta_2 &= \frac{C_{31} - C_{13}}{4}, & \beta_0\beta_3 &= \frac{C_{12} - C_{21}}{4}, \\ \beta_1\beta_2 &= \frac{C_{12} + C_{21}}{4}, & \beta_2\beta_3 &= \frac{C_{23} + C_{32}}{4}, & \beta_3\beta_1 &= \frac{C_{31} + C_{13}}{4}. \end{aligned}$$

Lastly, the MRP attitude representation is obtained as

$$\sigma_i = \frac{\beta_i}{1 + \beta_0}, \quad i \in \{1, 2, 3\}.$$

G. Numerical Attitude Simulator

To integrate the attitude of the LMO satellite we design a numerical integrator based on a RK4 scheme. Since the inertial positions and velocities of both LMO and GMO satellites have already been calculated, the propagated attitude state is $\vec{X} \in \mathbb{R}^6$:

$$\vec{X}(t) = \begin{bmatrix} \vec{\sigma}_{\mathcal{B}/\mathcal{N}}(t) \\ {}^B\vec{\omega}_{\mathcal{B}/\mathcal{N}}(t) \end{bmatrix}. \quad (24)$$

Since we have the initial state of the spacecraft $\vec{X}(t_0)$ in Section II.D, we only need differential equations to integrate the state. We express the differential equation for the angular velocity in the $\mathcal{B}$ frame because the inertia tensor is diagonal in that frame. Remember that the time derivative of an angular velocity is the same in all observers due to the Transport Theorem. We assume that the spacecraft is rigid and its dynamics obey:

$${}^B[I]{}^B\dot{\vec{\omega}}_{\mathcal{B}/\mathcal{N}} = -{}^B[\vec{\omega}_{\mathcal{B}/\mathcal{N}}]{}^B[I]{}^B\vec{\omega}_{\mathcal{B}/\mathcal{N}} + {}^B\vec{u}, \quad (25)$$

where ${}^B\vec{u}$ is an external control torque vector expressed in the $\mathcal{B}$ frame. Furthermore, the dynamics of the attitude MRP state $\vec{\sigma}_{\mathcal{B}/\mathcal{N}}$ is governed by the following differential equation:

$$\dot{\vec{\sigma}}_{\mathcal{B}/\mathcal{N}} = \frac{1}{4} [B(\vec{\sigma}_{\mathcal{B}/\mathcal{N}})]^B \vec{\omega}_{\mathcal{B}/\mathcal{N}} = \frac{1}{4} \left[(1 - \vec{\sigma}_{\mathcal{B}/\mathcal{N}}^T \vec{\sigma}_{\mathcal{B}/\mathcal{N}}) [I_{3 \times 3}] + 2[\vec{\sigma}_{\mathcal{B}/\mathcal{N}}] + 2\vec{\sigma}_{\mathcal{B}/\mathcal{N}} \vec{\sigma}_{\mathcal{B}/\mathcal{N}}^T \right]^B \vec{\omega}_{\mathcal{B}/\mathcal{N}} \quad (26)$$

To integrate the state $\vec{X}$, we implement a RK4 scheme with the following structure:

$$\dot{\vec{X}} = \vec{f}(t, \vec{X}, {}^B\vec{u}) = \begin{bmatrix} \dot{\vec{\sigma}}_{\mathcal{B}/\mathcal{N}} \\ {}^B\dot{\vec{\omega}}_{\mathcal{B}/\mathcal{N}} \end{bmatrix}, \quad (27)$$

where $\vec{f}$ defines the system dynamics, $\vec{X}$ is the state vector, and the time derivatives are determined by Eqs. 26 and 25. The pseu-decode for the RK4 scheme is detailed in Algorithm 1, including the shadow set switching logic for MRPs throughout the integration.

Algorithm 1 Pseudo-code RK4 Scheme

Inputs : $\vec{X}_0 = \begin{bmatrix} \vec{\sigma}_{\mathcal{B}/\mathcal{N}}(t_0) \\ \vec{\omega}_{\mathcal{B}/\mathcal{N}}(t_0) \end{bmatrix}$, $t_{\max}$, Δt , $t_n = 0$.
while $t_n < t_{\max}$ **do**
 Determine control torque ${}^{\mathcal{B}}\vec{u}$ with Eq 28 and Table 3;
 $\vec{X}_n = \vec{X}(t_n)$, ${}^{\mathcal{B}}\vec{u}_n = {}^{\mathcal{B}}\vec{u}(t_n)$;
 $\vec{k}_1 = \Delta t \vec{f}(\vec{X}_n, t_n, {}^{\mathcal{B}}\vec{u}_n)$;
 $\vec{k}_2 = \Delta t \vec{f}\left(\vec{X}_n + \frac{\vec{k}_1}{2}, t_n + \frac{\Delta t}{2}, {}^{\mathcal{B}}\vec{u}_n\right)$;
 $\vec{k}_3 = \Delta t \vec{f}\left(\vec{X}_n + \frac{\vec{k}_2}{2}, t_n + \frac{\Delta t}{2}, {}^{\mathcal{B}}\vec{u}_n\right)$;
 $\vec{k}_4 = \Delta t \vec{f}\left(\vec{X}_n + \vec{k}_3, t_n + \Delta t, {}^{\mathcal{B}}\vec{u}_n\right)$;
 $\vec{X}_{n+1} = \vec{X}_n + \frac{1}{6} (\vec{k}_1 + 2\vec{k}_2 + 2\vec{k}_3 + \vec{k}_4)$;
if $\|\vec{\sigma}_{\mathcal{B}/\mathcal{N}}\| > 1$ **then**
 map $\vec{\sigma}_{\mathcal{B}/\mathcal{N}}$ to shadow set: $\vec{\sigma}_{\mathcal{B}/\mathcal{N}}^s = -\frac{\vec{\sigma}_{\mathcal{B}/\mathcal{N}}}{\|\vec{\sigma}_{\mathcal{B}/\mathcal{N}}\|^2}$;
end if
 $t_{n+1} = t_n + \Delta t$;
 save spacecraft states $\vec{X}_n$ and ${}^{\mathcal{B}}\vec{u}_n$;
end while

Outputs: $t_i, \vec{X}_i, {}^{\mathcal{B}}\vec{u}_i, i \in \{0, 1, 2, \dots, N\}$

H. Attitude Control Law

In this work, we employ a simple proportional-derivative (PD) attitude control law of the form

$${}^{\mathcal{B}}\vec{u} = -K\vec{\sigma}_{\mathcal{B}/\mathcal{R}} - P{}^{\mathcal{B}}\vec{\omega}_{\mathcal{B}/\mathcal{R}}, \quad (28)$$

where $K \in \mathbb{R}^+$ is the feedback proportional gain, and $P \in \mathbb{R}^+$ is the feedback derivative gain. The goal of this control law is to drive the attitude tracking error $\vec{\sigma}_{\mathcal{B}/\mathcal{R}} = (\sigma_1, \sigma_2, \sigma_3)^T$ and the angular velocity tracking error ${}^{\mathcal{B}}\vec{\omega}_{\mathcal{B}/\mathcal{R}} = (\delta\omega_1, \delta\omega_2, \delta\omega_3)^T$ to zero, ensuring convergence of the body frame $\mathcal{B}$ to the desired reference frame $\mathcal{R}$. Throughout the mission, the reference frame $\mathcal{R} \in \{\mathcal{R}_s, \mathcal{R}_n, \mathcal{R}_c\}$ varies depending on the operational mode. To better understand the closed-loop behavior, we consider the nonlinear attitude dynamics introduced in Eqs. 25, 26, and 27. We linearize the dynamics about the equilibrium state $\vec{X} = \vec{0}$, corresponding to zero attitude and angular velocity tracking errors. The MRP kinematic equation simplifies to

$$\dot{\vec{\sigma}}_{\mathcal{B}/\mathcal{R}} \approx \frac{1}{4} {}^{\mathcal{B}}\vec{\omega}_{\mathcal{B}/\mathcal{R}}. \quad (29)$$

Because the body frame $\mathcal{B}$ is aligned with the principal axes of inertia, the inertia matrix is diagonal, and the rotational dynamics decouple into three independent systems. As a result, the linearized closed-loop dynamics for each principal axis can be written in state-space formulation:

$$\begin{bmatrix} \dot{\sigma}_i \\ \delta\dot{\omega}_i \end{bmatrix} = \begin{bmatrix} 0 & \frac{1}{4} \\ -\frac{K}{I_i} & -\frac{P}{I_i} \end{bmatrix} \begin{bmatrix} \sigma_i \\ \delta\omega_i \end{bmatrix}, \quad i \in \{1, 2, 3\}. \quad (30)$$

The roots of the corresponding characteristic equations can be expressed as:

$$\lambda_i = -\frac{1}{2I_i} \left(P \pm \sqrt{-KI_i + P^2} \right), \quad i \in \{1, 2, 3\}. \quad (31)$$

Therefore, the feedback gains can be used to determine whether the system is under-damped (complex roots), critically damped (double real root), or over-damped (two unique real roots).

The system presented in Eq 30 is analogous to three uncoupled second-order spring–mass–damper systems, where the inertia I_i plays the role of mass, the proportional gain K acts as a spring stiffness, and the derivative gain P provides damping. To prove it, we take the second time derivative of σ_i :

$$\ddot{\sigma}_i + \frac{P}{I_i} \dot{\sigma}_i + \frac{K}{4I_i} \sigma_i = 0, \quad (32)$$

which is directly comparable to the form of a second-order spring-mass-damper system:

$$\ddot{x} + 2\zeta_i \omega_{n_i} \dot{x} + \omega_{n_i}^2 x = 0, \quad (33)$$

where ω_{n_i} the natural frequency and ζ_i is the damping ratio. With this comparison, we can calculate the following values for each principal axis:

$$\omega_{n_i} = \frac{\sqrt{K}}{2\sqrt{I_i}}, \quad \zeta_i = \frac{P}{\sqrt{KI_i}}, \quad \tau_i = \frac{2I_i}{P}, \quad (34)$$

where τ_i , known as the time decay constant, is the time required for the tracking error to decrease to approximately $1/e$ of its initial value. The damping ratio ζ_i determines whether the response is under-damped ($\zeta_i < 1$), critically damped ($\zeta_i = 1$), or overdamped ($\zeta_i > 1$). The under-damped regime displays a damped oscillatory motion, while the critically damped regime represents the fastest decay without oscillation. We are not interested in over-damped regimes because they are slower decays than the other two regimes.

The feedback gains are selected to achieve desired control performances. Specifically, the slowest decay time is chosen to be $\tau_{\max} = 120$ s. Since the largest inertia corresponds to axis $i = 1$, the derivative gain is selected as:

$$P = \frac{2I_{\max}}{\tau_{\max}} = \frac{2I_1}{120} = \frac{1}{6} \text{ kg m}^2 \text{ s}^{-1}. \quad (35)$$

The corresponding decay time constants are:

$$\tau_1 = 120 \text{ s}, \quad \tau_2 = 60 \text{ s}, \quad \tau_3 = 90 \text{ s}. \quad (36)$$

In addition, one axis is chosen to be critically damped while the others remain under-damped. Enforcing $\zeta_{\max} = 1$ for the smallest inertia axis yields:

$$K = \frac{P^2}{I_{\min}} = \frac{P^2}{I_2} = \frac{1}{180} \text{ kg m}^2 \text{ s}^{-2}. \quad (37)$$

The resulting damping ratios are:

$$\zeta_1 \approx 0.707, \quad \zeta_2 = 1, \quad \zeta_3 \approx 0.816, \quad (38)$$

indicating that axis 2 is critically damped, while axes 1 and 3 exhibit under-damped responses. Since $\zeta_1 = \zeta_{\min}$ and $\tau_1 = \tau_{\max}$, the spacecraft reorients most slowly about the axis with the largest inertia (axis 1), while faster convergence occurs along the other axes. The critically damped axis (axis 2) exhibits the fastest non-oscillatory response. Overall, we have guaranteed that the slowest decay time constant in our system is 120 s, while forcing one component to be critically damped and the other two under-damped.

I. Mission Pointing Modes

Based on the control law presented in Eq 28, three specific pointing control modes are defined:

- Sun Pointing Control (SPC): aligns the spacecraft with the Sun direction to maximize the use of solar panels.
- Nadir Pointing Control (NPC): aligns the spacecraft with the Mars nadir direction to gather science data.
- GMO Pointing Control (GPC): aligns the LMO nano-satellite with the GMO spacecraft for communication purposes.

Each mode uses the same PD control structure, differing only in the reference frame $\mathcal{R}$. The appropriate mode is selected based on the spacecraft's orbital position and the relative geometry between the LMO nano-satellite, Mars, the Sun, and the GMO satellite.

When the spacecraft is on the sunlit side of Mars, it operates in Sun-pointing mode to ensure that the solar panels are oriented toward the Sun. When the spacecraft is on the shaded side, it switches to either science or communication mode. Communication mode is selected when the angular separation between the LMO and GMO position vectors is less than 35° . Otherwise, the spacecraft operates in nadir-pointing mode to perform scientific observations. Table 3 summarizes the pointing modes. The angle γ between the LMO and GMO position vectors is defined as

$$\cos \gamma(t) = \frac{{}^N\vec{r}_{LMO}(t) \cdot {}^N\vec{r}_{GMO}(t)}{|{}^N\vec{r}_{LMO}(t)| |{}^N\vec{r}_{GMO}(t)|}. \quad (39)$$

Table 3 Spacecraft Pointing Modes Summary

Mode	Reference Frame	Control Law	Condition
Sun	$\mathcal{R}_s$	${}^B\vec{u}_s = -K\vec{\sigma}_{B/\mathcal{R}_s} - P {}^B\vec{\omega}_{B/\mathcal{R}_s}$	${}^N\vec{r}_{LMO} \cdot \hat{n}_2 > 0$
Nadir	$\mathcal{R}_n$	${}^B\vec{u}_n = -K\vec{\sigma}_{B/\mathcal{R}_n} - P {}^B\vec{\omega}_{B/\mathcal{R}_n}$	${}^N\vec{r}_{LMO} \cdot \hat{n}_2 < 0, \gamma \geq 35^\circ$
GMO	$\mathcal{R}_c$	${}^B\vec{u}_c = -K\vec{\sigma}_{B/\mathcal{R}_c} - P {}^B\vec{\omega}_{B/\mathcal{R}_c}$	${}^N\vec{r}_{LMO} \cdot \hat{n}_2 < 0, \gamma < 35^\circ$

IV. Simulations & Discussion

Before combining all pointing modes into the full mission simulation, each control mode is developed and tested individually. For each case, a dedicated subsection analyzes a simplified scenario in which the spacecraft enters a given pointing mode at t_0 and remains in that mode for $t = 400$ s. After validating the performance of each control mode separately, the full mission scenario is simulated to evaluate the spacecraft's attitude pointing performance as it transitions between different control modes. Note that the initial attitude and angular velocity for all simulations are specified in Eqs. 2 and 3.

A. Sun Pointing Control Simulation

Figure 4 shows the time evolution of the spacecraft attitude and angular velocity during a SPC. The top plot exhibits a transition to the MRP shadow set at approximately $t \approx 170$ s, which ensures the MRP norm remains below unity without affecting the physical orientation. After this switch, the three components remain nearly constant, indicating convergence to a fixed inertial attitude. The bottom plot shows that all components of the angular velocity relative to the inertial frame decay to zero after approximately 75 s, confirming that the spacecraft reaches a non-rotating state since the reference frame $\mathcal{R}_s$ is inertially fixed.

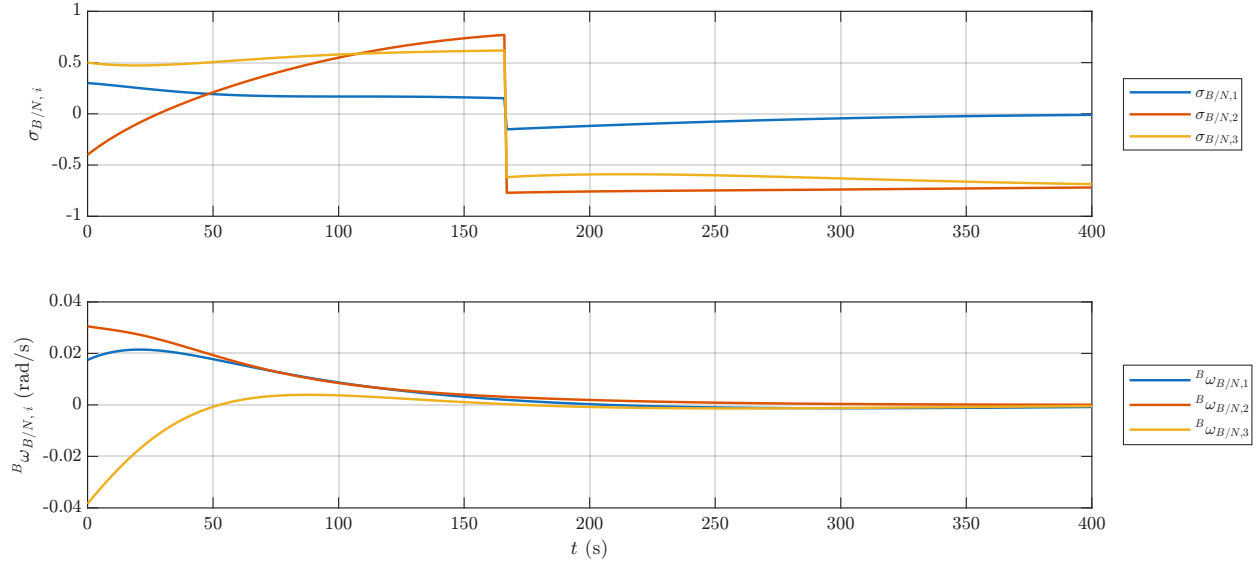


Fig. 4 Time evolution of spacecraft attitude and angular velocity during the SPC. Top: spacecraft attitude relative to the inertial frame, represented using MRPs. Bottom: spacecraft angular velocity relative to the inertial frame, expressed in the $\mathcal{B}$ frame.

Figure 5 presents the time evolution of the attitude tracking error, angular velocity tracking error, and control torque when the SPC is being applied. The attitude tracking error converges to zero, demonstrating asymptotic stability of the closed-loop system. The slowest convergence occurs along axis 1, consistent with the designed maximum time decay constant of $\tau_{\max} = \tau_1$. The damping behavior also matches the theoretical design: component 2 exhibits a critically damped response, while the components 1 and 3 display under-damped behavior. Similarly, the angular velocity tracking error decays to zero, confirming that rotational motion relative to the reference frame $\mathcal{R}_s$ is eliminated. The control torque is initially large but decreases smoothly to zero as the tracking errors vanish, consistent with the PD control law. Overall, the results confirm that the selected feedback gains yield the expected results for the SPC.

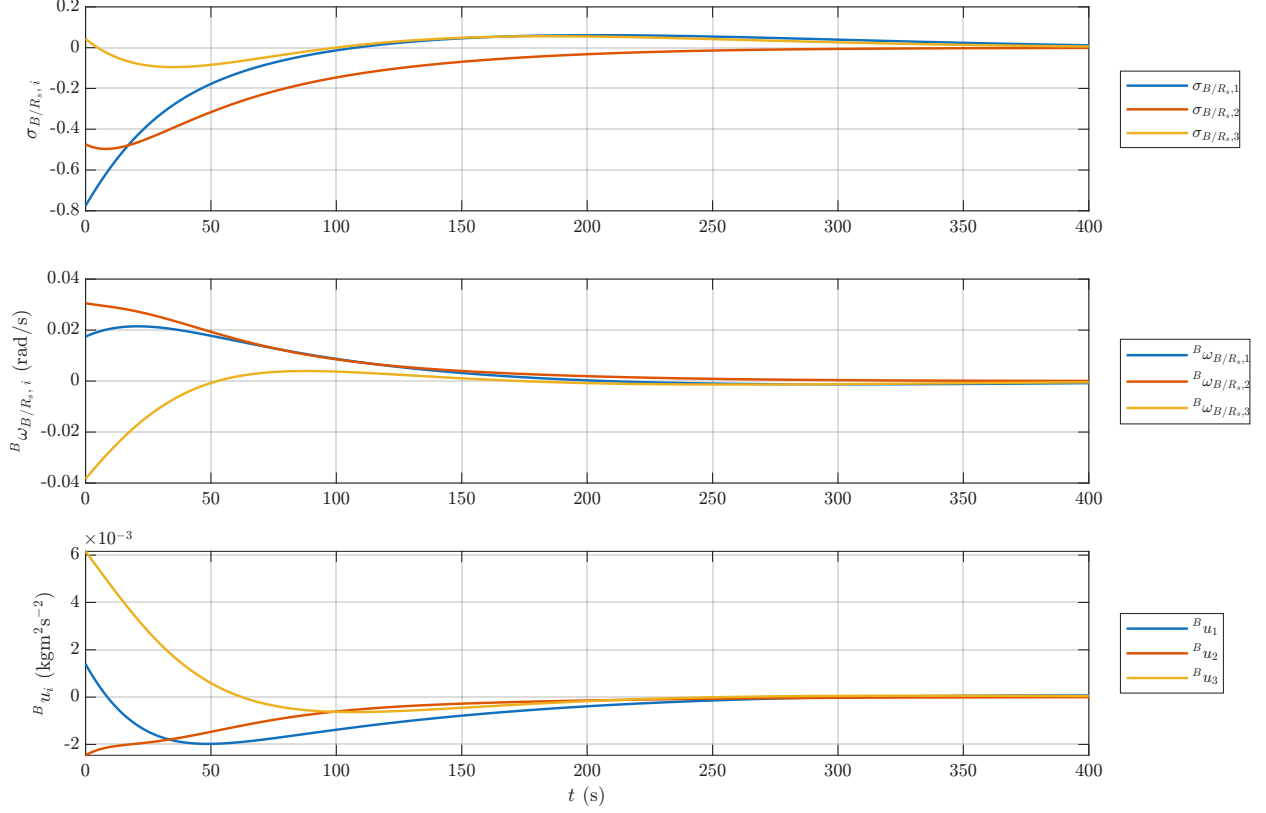


Fig. 5 Time evolution of tracking errors and control torque during the SPC. Top: attitude tracking error relative to frame $\mathcal{R}_s$, represented using MRPs. Middle: angular velocity tracking error relative to frame $\mathcal{R}_s$, expressed in the $\mathcal{B}$ frame. Bottom: control torque expressed in the $\mathcal{B}$ frame.

B. Nadir Pointing Control Simulation

The motion of the spacecraft during NPC is shown in Fig. 6. The attitude is represented using the original MRP set until approximately $t \approx 230$ s. After that, the second and third component of the MRP set remain constant while the first component slightly changes. Since component 1 has the slowest time decay constant, it takes longer for that component to be at the desired state.

Figure 7 displays the tracking errors and control torque during NPC. Both the attitude and angular velocity tracking errors asymptotically converge to zero. The top and middle subplots further illustrate the damping behavior: component 2 exhibits a critically damped response, while components 1 and 3 follow slightly under-damped regimes. Since components 1 and 3 are only lightly under-damped, their convergence rates remain close to that of component 2. The control torque decreases toward zero as the tracking errors vanish. Overall, the results confirm that the selected feedback gains provide stable and accurate tracking performance under NPC.

C. GMO Pointing Control Simulation

Unlike the previous control modes, Fig. 8 shows that the GPC does not cause the spacecraft attitude to switch to the MRP shadow set. Instead, the MRP components evolve continuously, reflecting the time-varying nature of the reference frame $\mathcal{R}_c$ associated with the GMO satellite. Figure 9 presents the time evolution of the tracking errors and control torque during GPC. Both the attitude and angular velocity tracking errors asymptotically converge to zero, confirming that the spacecraft successfully tracks the desired reference attitude and angular velocity. As the tracking errors vanish, the control torque also decreases toward zero, consistent with the PD control law.

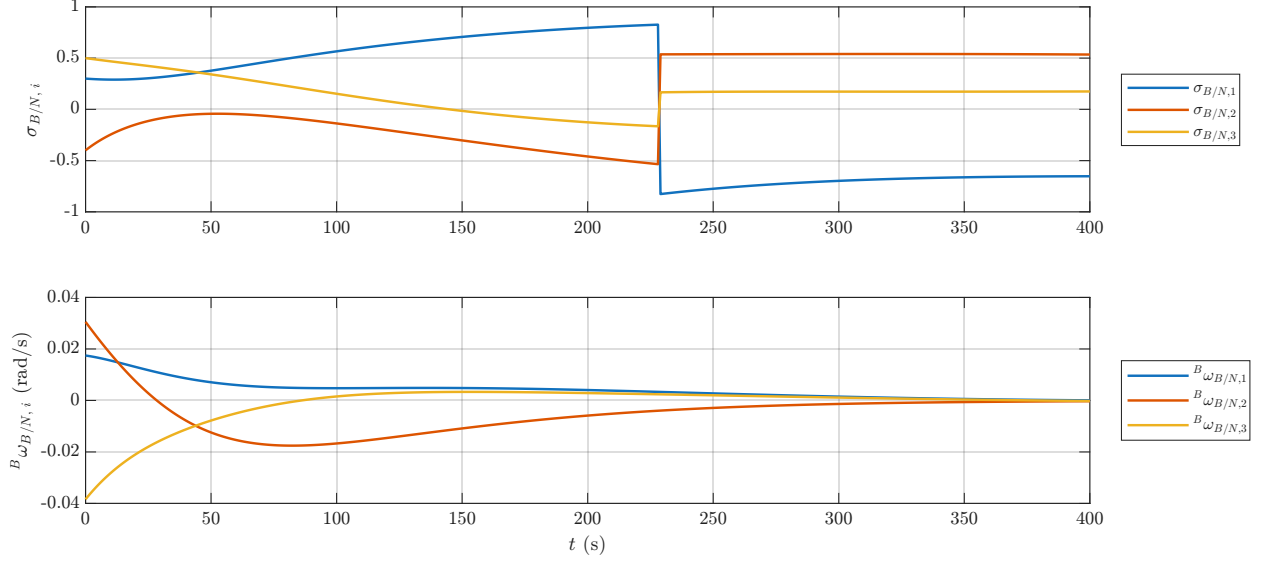


Fig. 6 Time evolution of spacecraft attitude and angular velocity during the NPC. Top: spacecraft attitude relative to the inertial frame, represented using MRPs. Bottom: spacecraft angular velocity relative to the inertial frame, expressed in the $\mathcal{B}$ frame.

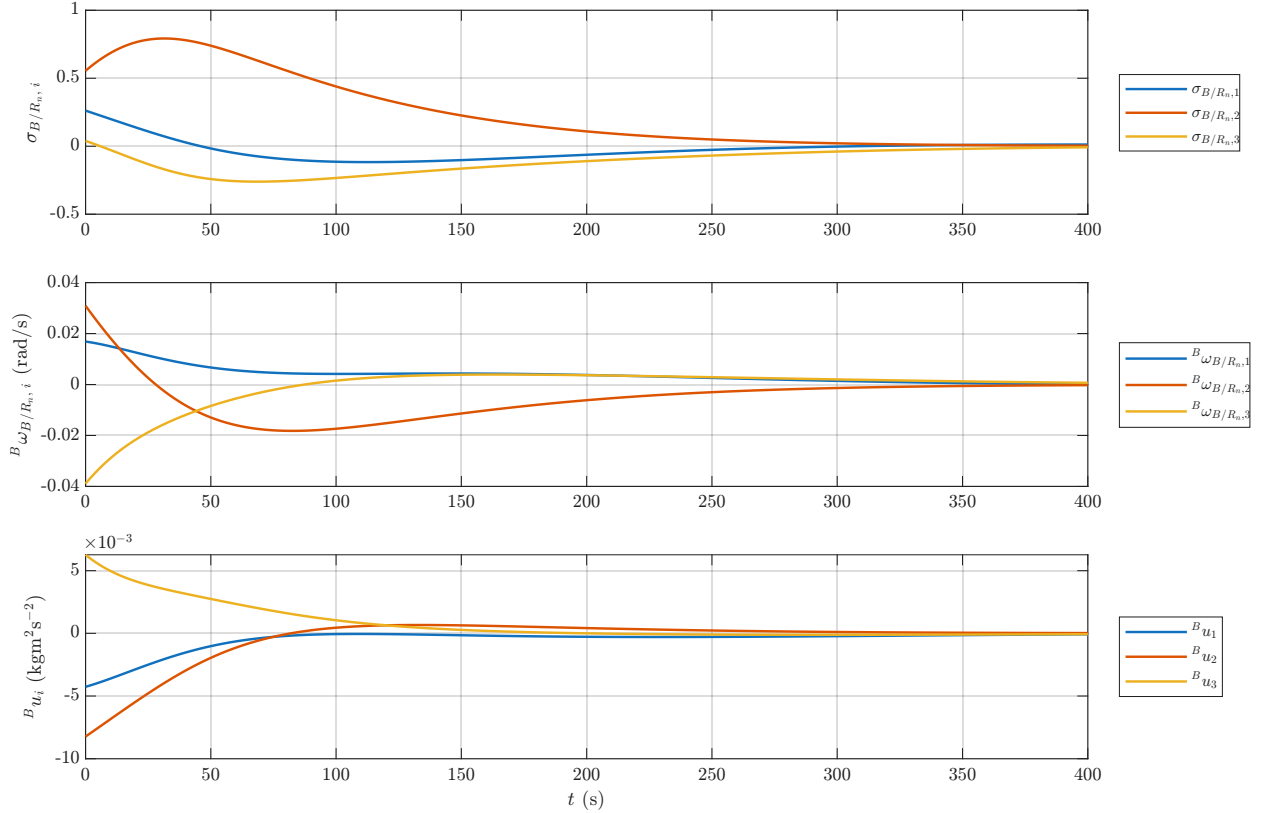


Fig. 7 Time evolution of tracking errors and control torque during the NPC. Top: attitude tracking error relative to frame $\mathcal{R}_n$, represented using MRPs. Middle: angular velocity tracking error relative to frame $\mathcal{R}_n$, expressed in the $\mathcal{B}$ frame. Bottom: control torque expressed in the $\mathcal{B}$ frame.

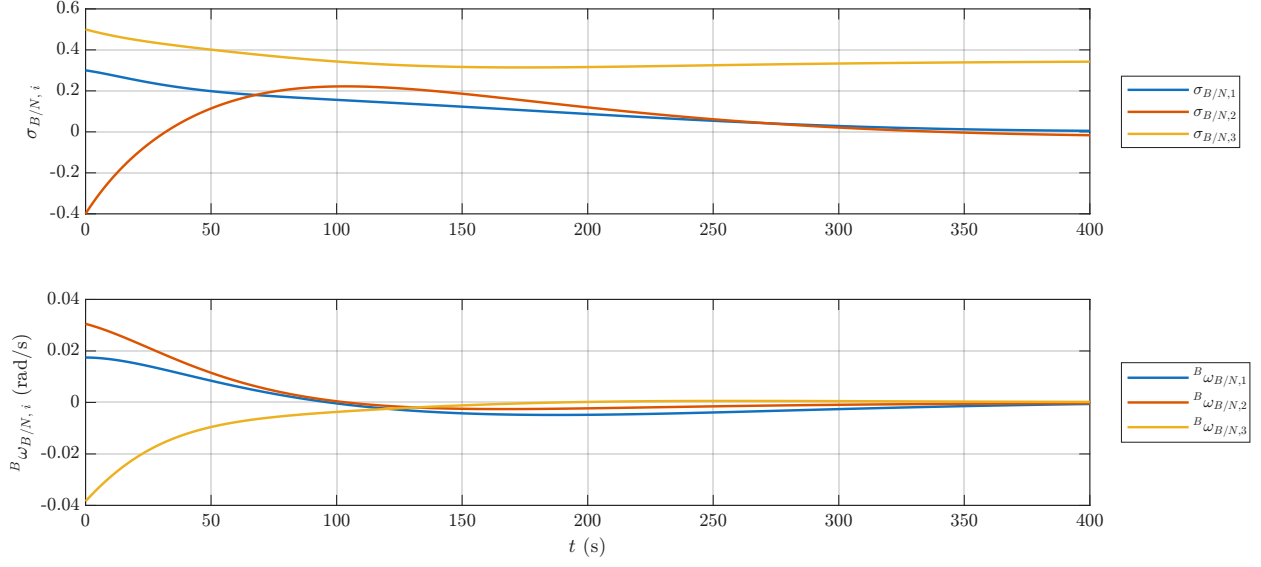


Fig. 8 Time evolution of spacecraft attitude and angular velocity during the GPC. Top: spacecraft attitude relative to the inertial frame, represented using MRPs. Bottom: spacecraft angular velocity relative to the inertial frame, expressed in the $\mathcal{B}$ frame.

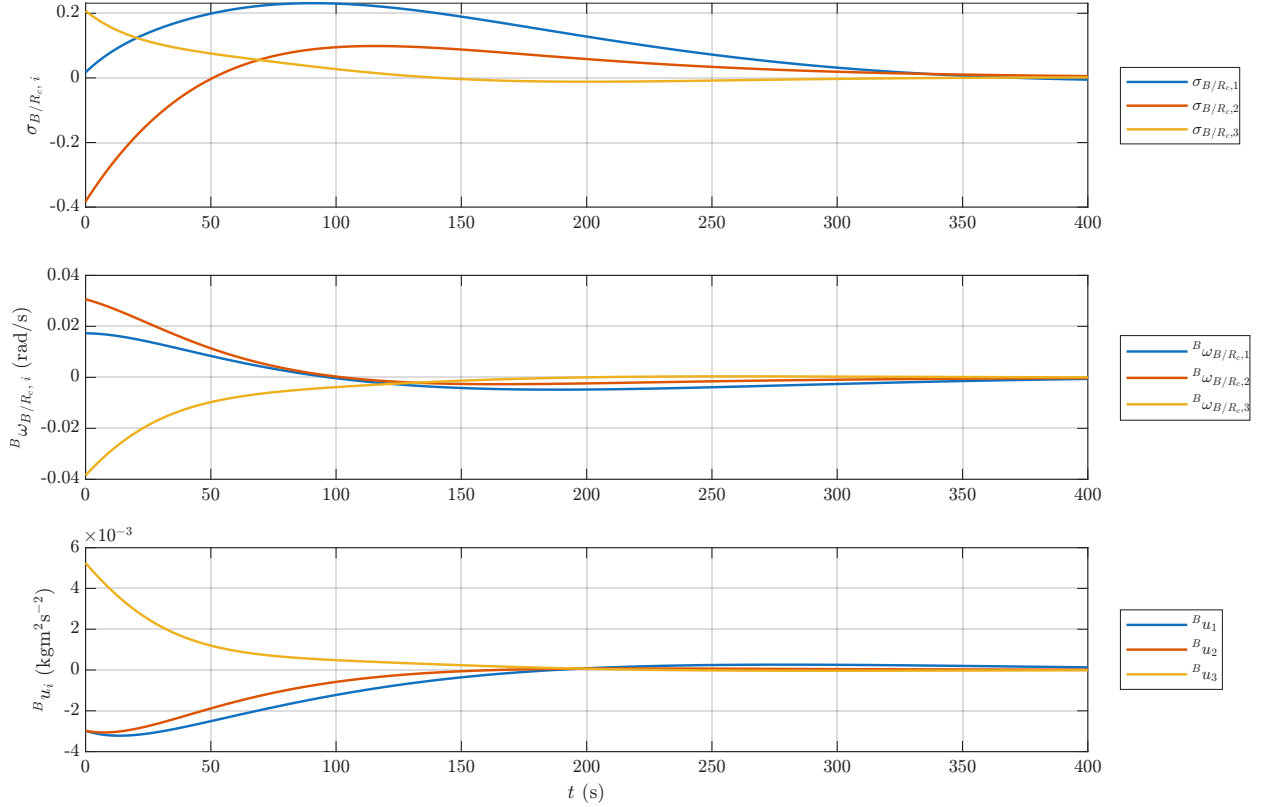


Fig. 9 Time evolution of tracking errors and control torque during the NPC. Top: attitude tracking error relative to frame $\mathcal{R}_c$, represented using MRPs. Middle: angular velocity tracking error relative to frame $\mathcal{R}_c$, expressed in the $\mathcal{B}$ frame. Bottom: control torque expressed in the $\mathcal{B}$ frame.

D. Mission Simulation

For the mission scenario simulation, the initial conditions in Eqs . 2 and 3. are propagated in time through 6500 s. Throughout the mission, the spacecraft autonomously switches modes according to the relative positions of both spacecrafts, as detailed in Table 3. Note that to calculate the inertial positions of the spacecrafts, we use the initial orbit conditions in Table 1 and the expression for $\theta(t)$ in Eq. 9. As a result, Fig. 10 displays how the pointing mode evolves throughout the whole mission. The Sun-pointing mode is enforced during the time interval $(0, 1918) \cup (5469, 6500)$ s, the Nadir-pointing mode is enforced during $(1918, 3057) \cup (4067, 5469)$ s, and the GMO-pointing mode is enforced only during $(3057, 4067)$ s. Therefore, we have successfully implemented mode-switching during the mission scenario simulation, where the spacecraft is able to complete all the necessary functions in order for the mission goals to be accomplished.

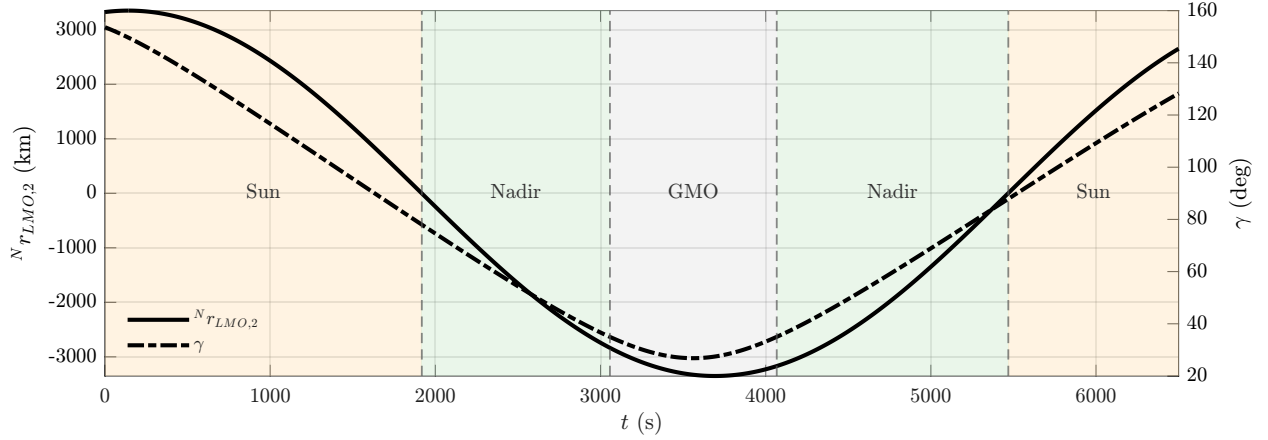


Fig. 10 Time evolution of the pointing modes during the proposed mission. The left y axis and the solid line represent the second component of the inertial position of the LMO satellite. The right y axis and the dashed line represent the angle γ between the LMO satellite and the GMO satellite. Each shaded region represents a different pointing mode, as the text label indicates. Different pointing modes are separated by thinner dashed lines.

Figure 11 depicts the evolution of the spacecraft's attitude and angular velocity with respect to the inertial frame throughout the whole mission. During the Sun-pointing time intervals (yellow regions), the attitude MRP state switches to the shadow set several times. This means that the spacecraft maintains a nearly constant attitude corresponding to a rotation near 180° , and small numerical variations lead the MRP set to repeatedly switch to the shadow set. During the Nadir-pointing time intervals (green regions), the MRP components continuously change to always point to the center of Mars. The same can be said for the GMO-pointing time intervals (gray regions), as the LMO spacecraft needs to continuously adapt to the new position of the mothercraft.

In addition, the angular velocity of the spacecraft with respect to the inertial frame remains small except during transitions between pointing modes, where brief increases are observed. This behavior arises because the reference frames themselves rotate slowly with respect to the inertial frame, so only a small angular velocity is required for the spacecraft to maintain alignment.

Figure 12 shows the time evolution of the attitude and angular velocity tracking errors relative to the active reference frame, as well as the applied control torque. The tracking errors remain close to zero throughout the mission, except during pointing mode changes, where brief deviations are observed due to the change in reference frame. These deviations are quickly corrected, indicating that the spacecraft is able to accurately reorient and maintain the desired pointing conditions. A similar behavior is observed in the angular velocity tracking error, which remains near zero during steady operation and only deviates during mode switches. As both the attitude and angular velocity tracking errors converge to zero, the control torque correspondingly decreases to zero, as expected from the PD control law. Overall, these results demonstrate that the spacecraft successfully achieves the desired pointing objectives throughout the mission.

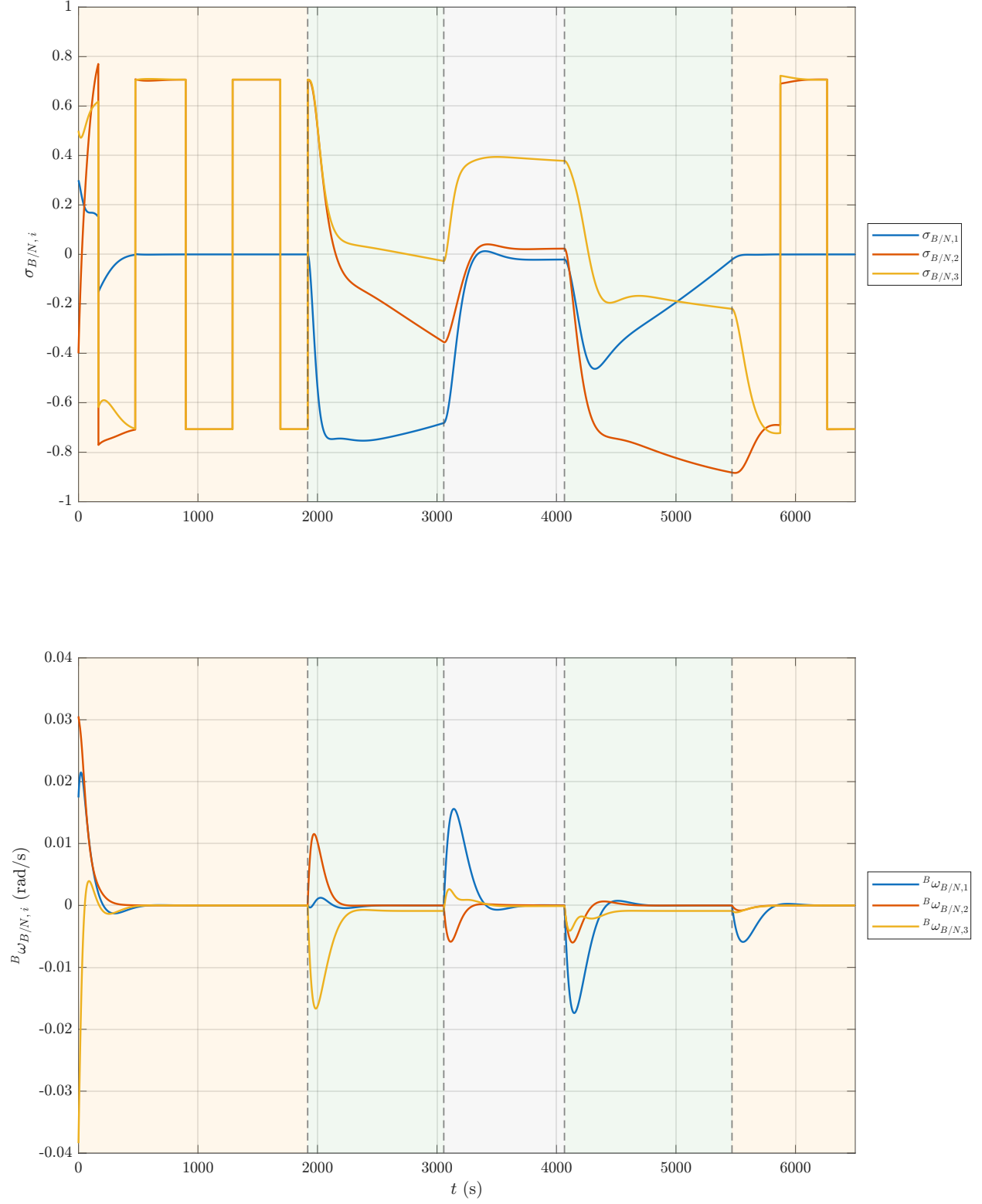


Fig. 11 Time evolution of spacecraft attitude and angular velocity during the proposed mission. Top: spacecraft attitude relative to the inertial frame, represented using MRPs. Bottom: spacecraft angular velocity relative to the inertial frame, expressed in the $\mathcal{B}$ frame. The shaded regions and the dashed lines are used to represent different pointing modes.

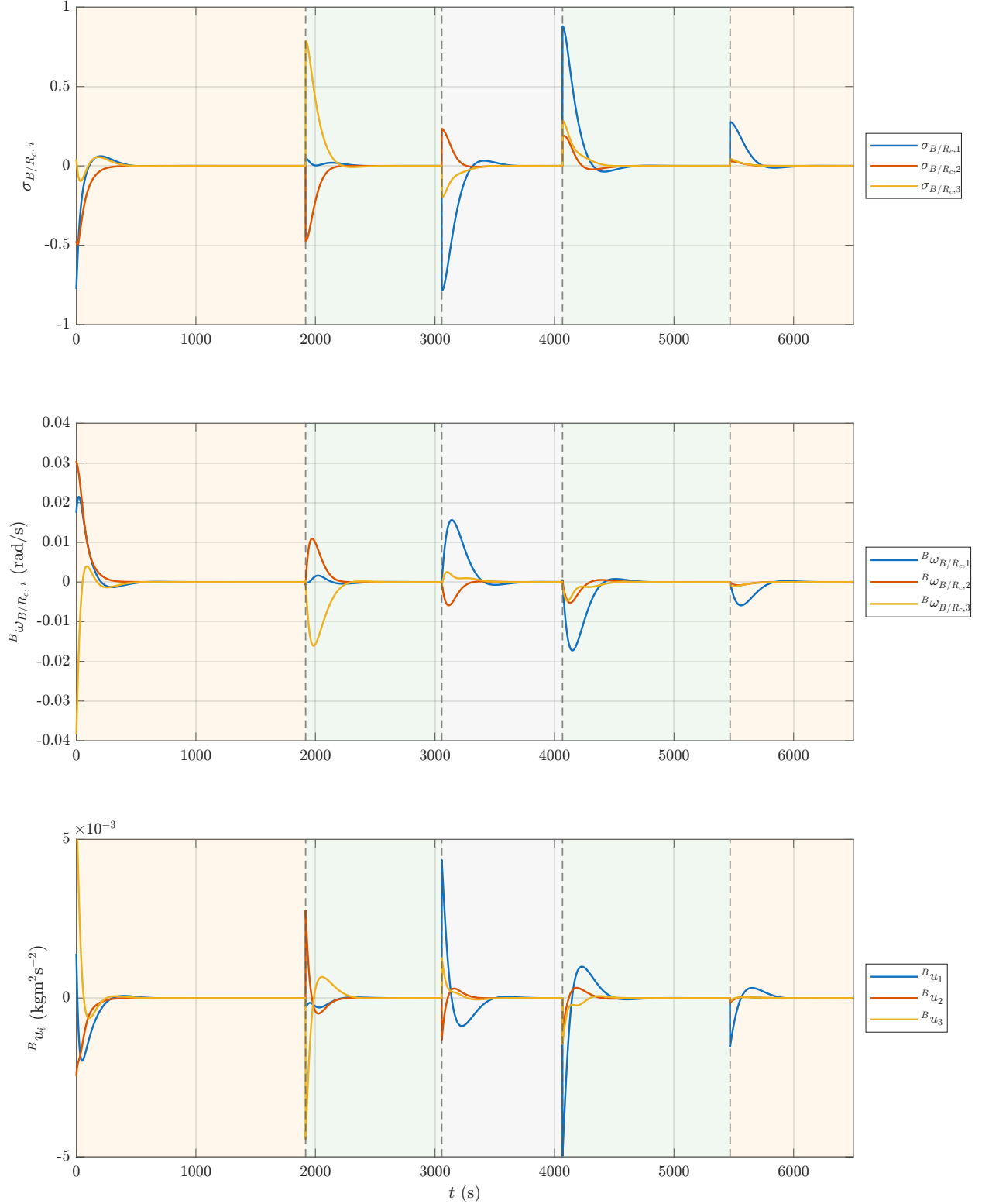


Fig. 12 Time evolution of tracking errors and control torque during the proposed mission. Top: attitude tracking error relative to the active reference frame, represented using MRPs. Second: angular velocity tracking error relative to the active reference frame, expressed in the $\mathcal{B}$ frame. Third: control torque expressed in the $\mathcal{B}$ frame. Bottom: mission geometry and operating modes as functions of the second component of the spacecraft inertial position and the angle γ between the two spacecrafts. The shaded regions and the dashed lines are used to represent different pointing modes.

V. Conclusion

This work presented the modeling, simulation, and control of a nano-satellite in Low Mars Orbit subject to multiple pointing requirements, including Sun-pointing, nadir-pointing, and communication with a mother spacecraft in Geosynchronous Mars Orbit. Each pointing mode was independently tested and shown to achieve accurate attitude tracking and asymptotic stability under a proportional-derivative control law. These modes were then integrated into an autonomous mode-switching framework based on mission geometry, enabling the spacecraft to transition between operational requirements throughout the mission. The full mission simulation demonstrates that the spacecraft consistently satisfies the desired pointing conditions, validating our autonomous mode-switching framework.

References

- [1] Schaub, H., and Junkins, J. L., *Analytical Mechanics of Space Systems*, 4th ed., AIAA, Reston, VA, 2018.

Appendix

```

1 Global
2 Set up global parameters
3 % LMO satellite struct
4 LMO = struct();
5 LMO.r_Mars = 3396.19; % radius of Mars in km
6 LMO.h = 400; % altitude in km
7 LMO.r = LMO.r_Mars+ LMO.h; % orbit radius in km
8 LMO.theta_dot = 0.000884797; % orbit rate in rad/s
9 LMO.Omega = deg2rad(20); % argument of periapsis in rad
10 LMO.i = deg2rad(30); % inclination in rad
11 LMO.theta_t0 = deg2rad(60); % initial true anomaly in rad
12
13 % GMO satellite struct
14 GMO = struct();
15 GMO.r = 20424.2; % orbit radius in km
16 GMO.theta_dot = 0.0000709003; % orbit rate in rad/s
17 GMO.Omega = 0; % argument of periapsis in rad
18 GMO.i = 0; % inclination in rad
19 GMO.theta_t0 = deg2rad(250); % initial true anomaly in rad
20
21 % Struct with both LMO and GMO structs into a martian orbits struct
22 MOs = struct();
23 MOs.LMO = LMO;
24 MOs.GMO = GMO;
25
26 % Define spacecraft object
27 SC = struct();
28 SC.B_I_c = [10, 0, 0;
29             0, 5, 0;
30             0, 0, 7.5]; % inertia tensor in body frame B at center of mass c in kgm^2
31 SC.B_wBN_vec_t0 = [deg2rad(1); deg2rad(1.75); deg2rad(-2.2)]; % initial body angular
32                     velocity in rad/s
33 SC.sigmaBN_vec_t0 = [0.3 ; -0.4; 0.5]; % initial attitude
34 SC.X0_vec = [0.3 ; -0.4; 0.5; deg2rad(1); deg2rad(1.75); deg2rad(-2.2)]; % initial
35                     state
36
37 % Define control params
38 ctrl_PD = struct();
39 ctrl_PD.K = 1/180; % feedback gain in kgm^2/s^2
40 ctrl_PD.P = 1/6; % feedback gain in kgm^2/s
41 ctrl_PD.mode = 'Sun'; % it can be sun pointing mode, nadir pointing mode or
42                     communication pointing mode
43
44 % Generalized parameters
45 params_mission = struct();
46 params_mission.LMO = LMO;
47 params_mission.GMO = GMO;
48 params_mission.SC = SC;
49 params_mission.ctrl = ctrl_PD;
50
51 Task 1: Orbit Simulation
52 Functions
53 % function [N_r_vec, N_v_vec] = inertial_state(r, theta_dot, E313_vec)
54 % %=====
55 % %
56 % % Calculates inertial position vector and inertial velocity vector.

```

```

55 % % Circular orbit with constant radius and angular velocity is assumed.
56 % % Uses Dr. Schaub's rigid kinematics toolbox.
57 % % Refer to project pdf for N,O frame definitions and formulas used.
58 % %
59 % % Author: G. Montseny
60 % % Date: March 31, 2026
61 % %
62 % %
63 % % INPUT:           Description           Units
64 % %
65 % %   r           -      radius of circular orbit           km
66 % %   theta_dot   -      angular velocity of circular orbit   rad/s
67 % %   E313_vec    -      (3-1-3) Euler angles vector         rad
68 % %
69 % % OUTPUT:
70 % %
71 % %   N_r_vec      -      inertial position (3x1)             km
72 % %   N_v_vec      -      inertial velocity (3x1)             km/s
73 % %=====
74 %
75 %   % Initialization & Inputs
76 %   E313_vec = E313_vec(:); % column shape
77 %
78 %   % Compute inertial vectors in O frame
79 %   O_r_vec = [r;0;0];
80 %   O_v_vec = [0;r*theta_dot;0];
81 %
82 %   % Compute DCM [NO]
83 %   ON = Euler3132C(E313_vec);
84 %   NO = ON';
85 %
86 %   % Compute inertial vectors in N frame
87 %   N_r_vec = NO*O_r_vec;
88 %   N_v_vec = NO*O_v_vec;
89 % end
90
91
92
93 function theta_t = theta(theta_t0, theta_dot, t)
94 %=====
95 %
96 % Author: G. Montseny
97 % Date: March 31, 2026
98 %
99 %
100 % INPUT:           Description           Units
101 %
102 %   theta_t0      -      true anomaly at initial time t0       rad
103 %   theta_dot     -      angular velocity of circular orbit     rad/s
104 %   t             -      time                                     s
105 %
106 % OUTPUT:
107 %
108 %   theta_t       -      true anomaly at final time t           rad/s
109 %=====
110
111 % Formula
112 theta_t = theta_t0 + theta_dot*t;
113

```

```

114 end
115
116 function [N_r_vec, N_v_vec] = inertial_state(params_orb, t)
117 %=====
118 %
119 % Calculates inertial position vector and inertial velocity vector.
120 % Circular orbit with constant radius and angular velocity is assumed.
121 % Uses Dr. Schaub's rigid kinematics toolbox.
122 % Refer to project pdf for N,0 frame definitions and formulas used.
123 %
124 % Author: G. Montseny
125 % Date: March 31, 2026
126 %
127 %
128 % INPUT:          Description          Units
129 %
130 % params_orb    -   orbital parameters  (LMO/GMO)          -
131 % t              -   time                  S
132 %
133 % OUTPUT:
134 %
135 % N_r_vec       -   inertial position (3x1)                km
136 % N_v_vec       -   inertial velocity (3x1)                km/s
137 %=====
138
139 % Initialization & Inputs
140 r = params_orb.r;
141 theta_dot = params_orb.theta_dot;
142 theta_t0 = params_orb.theta_t0;
143
144 % Compute theta_t and Euler vector
145 theta_t = theta(theta_t0, theta_dot, t);
146 E313_vec = [params_orb.Omega; params_orb.i; theta_t];
147
148 % Compute inertial vectors in 0 frame
149 O_r_vec = [r;0;0];
150 O_v_vec = [0;r*theta_dot;0];
151
152 % Compute DCM [NO]
153 ON = Euler3132C(E313_vec);
154 NO = ON';
155
156 % Compute inertial vectors in N frame
157 N_r_vec = NO*O_r_vec;
158 N_v_vec = NO*O_v_vec;
159
160 end
161 Code
162 LMO
163 % theta_t = theta(params.LMO.theta_t0, params.LMO.theta_dot, 450);
164 % E313_vec = [params.LMO.Omega; params.LMO.i; theta_t];
165 % [N_rLMO_vec, N_vLMO_vec] = inertial_state(params.LMO.r, params.LMO.theta_dot,
166 % E313_vec);
167 [N_rLMO_vec, N_vLMO_vec] = inertial_state(params.LMO, 450)
168 GMO
169 % theta_t = theta(params.GMO.theta_t0, params.GMO.theta_dot, 1150);
170 % E313_vec = [params.GMO.Omega; params.GMO.i; theta_t];
171 % [N_rGMO_vec, N_vGMO_vec] = inertial_state(params.GMO.r, params.GMO.theta_dot,
172 % E313_vec);
173 [N_rGMO_vec, N_vGMO_vec] = inertial_state(params.GMO, 1150)

```

```

171 Submission formatting
172 file = fopen(fullfile(outputFolder,'Task1_1_v2.txt'),'w');
173 fprintf(file, '%.5f_%.5f_%.5f', N_rLMO_vec);
174 fclose(file);
175
176 file = fopen(fullfile(outputFolder,'Task1_2_v2.txt'),'w');
177 fprintf(file, '%.5f_%.5f_%.5f', N_vLMO_vec);
178 fclose(file);
179
180 file = fopen(fullfile(outputFolder,'Task1_3_v2.txt'),'w');
181 fprintf(file, '%.5f_%.5f_%.5f', N_rGMO_vec);
182 fclose(file);
183
184 file = fopen(fullfile(outputFolder,'Task1_4_v2.txt'),'w');
185 fprintf(file, '%.5f_%.5f_%.5f', N_vGMO_vec);
186 fclose(file);
187
188 Task 2: Orbit Frame Orientation
189 Functions
190 function HN = N2H(params_orb, t)
191 %=====
192 %
193 % Calculates [HN] DCM for time t.
194 % For LMO or GMO.
195 %
196 % Author: G. Montseny
197 % Date: March 31, 2026
198 %
199 %
200 % INPUT:                Description                Units
201 %
202 %   params_orb   -   orbital parameters   (LMO/GMO)   -
203 %   t            -   time                  S
204 %
205 % OUTPUT:
206 %
207 %   HN           -   DCM from N frame to H frame (3x3)   -
208 %=====
209
210 % Calculate true anomaly
211 theta_t = theta(params_orb.theta_t0, params_orb.theta_dot, t);
212
213 % Extract Euler angles column vector
214 E313_vec = [params_orb.Omega; params_orb.i; theta_t];
215
216 % Compute DCM [HN]
217 HN = Euler3132C(E313_vec);
218 end
219
220
221 Code
222 HN_300 = N2H(params.LMO, 300)
223 Submission Formatting
224 file = fopen(fullfile(outputFolder,'Task2.txt'),'w');
225 fprintf(file, '%.5f_%.5f_%.5f_%.5f_%.5f_%.5f_%.5f_%.5f', HN_300');
226 fclose(file);
227
228 Task 3: Sun-Pointing Reference Frame Orientation
229 Functions

```

```

230 function RsN = N2Rs()
231 %=====
232 %
233 % Calculates RsN DCM, where Rs is solar-facing reference frame.
234 %
235 % Author: G. Montseny
236 % Date: March 31, 2026
237 %
238 %
239 % INPUT:           Description           Units
240 %
241 % t               -           time               s
242 %
243 % OUTPUT:
244 %
245 % RsN             -           DCM from N frame to Rs frame (3x3)           -
246 %=====
247
248 RsN = [-1,0,0;
249        0,0,1;
250        0,1,0];
251 end
252
253
254 Code
255 Submission Formatting
256 file = fopen(fullfile(outputFolder,'Task3_1.txt'),'w');
257 fprintf(file, '%.5f_%.5f_%.5f_%.5f_%.5f_%.5f_%.5f_%.5f', N2Rs');
258 fclose(file);
259 file = fopen(fullfile(outputFolder,'Task3_2.txt'),'w');
260 fprintf(file, '%.5f_%.5f_%.5f', [0;0;0]);
261 fclose(file);
262
263 Task 4: Nadir-Pointing Reference Frame Orientation
264 Functions
265 function RnN = N2Rn(params_orb, t)
266 %=====
267 %
268 % Calculates [RnN] DCM for time t.
269 % Spacecraft sensor points towards the center of Mars/nadir direction.
270 % For LMO or GMO.
271 %
272 % Author: G. Montseny
273 % Date: March 31, 2026
274 %
275 %
276 % INPUT:           Description           Units
277 %
278 % params_orb      -           orbital parameters   (LMO/GMO)           -
279 % t               -           time               s
280 %
281 % OUTPUT:
282 %
283 % RnN             -           DCM from N frame to Rn frame (3x3)           -
284 %=====
285
286 % Compute RnH
287 RnH = [-1,0,0;
288        0,1,0;

```

```

289         0,0,-1];
290
291     % Compute HN
292     HN = N2H(params_orb, t);
293
294     % Compute RnN
295     RnN = RnH*HN;
296 end
297
298 function N_w_RnN_vec = inertial_w_RnN(params_orb, t)
299 %=====
300 %
301 % Calculates N_w_RnN_vec for time t.
302 % It's the angular velocity of frame Rn wrt to inertial frame N,
303 % expressed (coordinated) in N frame coordinates.
304 % For LMO or GMO.
305 %
306 % Author: G. Montseny
307 % Date: March 31, 2026
308 %
309 %
310 % INPUT:                Description                Units
311 %
312 % params_orb    -    orbital parameters    (LMO/GMO)    -
313 % t              -    time                    s
314 %
315 % OUTPUT:
316 %
317 % N_w_RnN_vec    -    angular velocity of Rn frame wrt to N frame    rad/s
318 %=====
319
320     % Compute NH
321     HN = N2H(params_orb, t);
322     NH = HN';
323
324     % Final formula
325     N_w_RnN_vec = NH*[0;0;params_orb.theta_dot];
326 end
327
328
329 Code
330 RnN_330 = N2Rn(LMO, 330);
331 N_w_RnN_vec_330 = inertial_w_RnN(LMO, 330);
332 Submission Formatting
333 file = fopen(fullfile(outputFolder,'Task4_1.txt'),'w');
334 fprintf(file, '%.5f_%.5f_%.5f_%.5f_%.5f_%.5f_%.5f_%.5f', RnN_330');
335 fclose(file);
336
337 file = fopen(fullfile(outputFolder,'Task4_2.txt'),'w');
338 fprintf(file, '%.9f_%.9f_%.9f', N_w_RnN_vec_330);
339 fclose(file);
340
341 Task 5: GMO - Pointing Reference Frame Orientation
342 Functions
343 function RcN = N2Rc(params_glob, t)
344 %=====
345 %
346 % Calculates [RcN] DCM for time t.
347 % Communications mode.

```



```

348 %
349 % Author: G. Montseny
350 % Date: March 31, 2026
351 %
352 %
353 % INPUT:          Description          Units
354 %
355 %   params_glob    -      global parameters of the problem      -
356 %   t              -      time                                   s
357 %
358 % OUTPUT:
359 %
360 %   RcN            -      DCM from N frame to Rc frame (3x3)      -
361 %=====
362
363 % Access parameters for both orbits
364 params_LMO = params_glob.LMO;
365 params_GMO = params_glob.GMO;
366
367 % Compute orbit DCMs for LMO and GMO
368 HN_LMO = N2H(params_LMO, t); NH_LMO = HN_LMO';
369 HN_GMO = N2H(params_GMO, t); NH_GMO = HN_GMO';
370
371 % Compute N_r1_hat
372 N_rLMO_vec = NH_LMO*[params_LMO.r;0;0];
373 N_rGMO_vec = NH_GMO*[params_GMO.r;0;0];
374
375 N_Delta_r_vec = N_rGMO_vec - N_rLMO_vec;
376
377 N_r1_hat = - N_Delta_r_vec/norm(N_Delta_r_vec);
378
379 % Compute N_r2_hat
380 N_r2_vec = cross(N_Delta_r_vec,[0;0;1]);
381 N_r2_hat = N_r2_vec/norm(N_r2_vec);
382
383 % Compute N_r3_hat
384 N_r3_hat = cross(N_r1_hat,N_r2_hat);
385
386 % Compute DCM
387 RcN = [N_r1_hat';N_r2_hat';N_r3_hat'];
388
389 end
390
391 function N_w_RcN_vec = inertial_w_RcN(params_glob, t)
392 %=====
393 %
394 % Calculates N_w_RcN_vec for time t.
395 % It's the angular velocity of frame Rc wrt to inertial frame N,
396 % expressed (coordinatized) in N frame coordinates.
397 % Numerical differences are used to compute the derivative of RcN
398 %
399 % Author: G. Montseny
400 % Date: March 31, 2026
401 %
402 %
403 % INPUT:          Description          Units
404 %
405 %   params_glob    -      global parameters of the problem      -
406 %   t              -      time                                   s

```

```

407 %
408 % OUTPUT:
409 %
410 % N_w_RnN_vec - angular velocity of Rn frame wrt to N frame rad/s
411 %=====
412
413 % Compute RcN
414 RcN_t = N2Rc(params_glob, t);
415
416 % Compute Rc_wRcN_tilde through numerical differences
417 dt = 1e-5;
418 RcN_tpdtdt = N2Rc(params_glob, t+dt);
419
420 if t > dt
421
422     % Central difference
423     RcN_tmdtdt = N2Rc(params_glob, t-dt);
424     RcN_dot = (RcN_tpdtdt - RcN_tmdtdt)/(2*dt);
425
426 else
427     % Forward difference
428     RcN_dot = (RcN_tpdtdt - RcN_t)/dt;
429 end
430
431 Rc_wRcN_tilde = - RcN_dot*RcN_t';
432
433 % Change to inertial frame
434 N_wRcN_tilde = RcN_t'*Rc_wRcN_tilde*RcN_t;
435
436 % Extract vector from skew operator
437 N_w_RcN_vec = [N_wRcN_tilde(3,2); N_wRcN_tilde(1,3); N_wRcN_tilde(2,1)];
438 end
439 Code
440 RcN_330 = N2Rc(params, 330);
441 N_w_RcN_vec_330 = inertial_w_RcN(params, 330);
442 Submission Formatting
443 file = fopen(fullfile(outputFolder,'Task5_1.txt'),'w');
444 fprintf(file, '%.5f_%.5f_%.5f_%.5f_%.5f_%.5f_%.5f_%.5f_%.5f', RcN_330');
445 fclose(file);
446
447 file = fopen(fullfile(outputFolder,'Task5_2.txt'),'w');
448 fprintf(file, '%.8f_%.8f_%.8f', N_w_RcN_vec_330);
449 fclose(file);
450
451 Task 6: Attitude Error Evaluation
452 Functions
453 function [sigma_BR_vec, B_w_BR_vec] = track_err(sigma_BN_vec, B_w_BN_vec, RN,
454     N_w_RN_vec)
455 %=====
456 %
457 % Calculates associated tracking errors at time t.
458 % Tracking errors between body frame B and reference frame R.
459 % N frame is the inertial frame.
460 % All inputs and outputs are given at time t.
461 %
462 % Author: G. Montseny
463 % Date: April 1, 2026
464 %

```

```

465 % INPUT:           Description           Units
466 %
467 % sigma_BN_vec     MRP vector B frame wrt N frame (3x1)      -
468 % B_w_BN_vec       angular speed of frame B wrt to frame N,  rad/s
469 %                   coordinatized in B frame (3x1)
470 % NR               DCM from N frame to B frame (3x3)         -
471 % N_w_RN_vec       angular speed of frame R wrt to frame N,  rad/s
472 %                   coordinatized in N frame (3x1)
473 %
474 %
475 % OUTPUT:
476 %
477 % sigma_BR_vec     MRP vector B frame wrt R frame (3x1)      -
478 % B_w_BR_vec       angular speed of frame B wrt to frame R,  rad/s
479 %                   coordinatized in B frame (3x1)
480 %=====
481
482 % Initialization
483 sigma_BN_vec = sigma_BN_vec(:);
484 B_w_BN_vec = B_w_BN_vec(:);
485 N_w_RN_vec = N_w_RN_vec(:);
486
487 % Calculate [BN](t)
488 BN = MRP2C(sigma_BN_vec);
489
490 % Calculate B_w_BR_vec(t)
491 B_w_BR_vec = B_w_BN_vec - BN*N_w_RN_vec;
492
493 % Calculate sigma_BR_vec(t)
494 BR = BN*RN';
495 sigma_BR_vec = C2MRP(BR);
496
497 end
498
499 Code
500 % Body Frame B Inputs
501 sigma_BN_vec_t0 = [0.3; -0.4; 0.5];
502 B_w_BN_vec_t0 = [deg2rad(1); deg2rad(1.75); deg2rad(-2.2)]; % rad/s
503
504 % Sun-Pointing frame Rs
505 RsN_t0 = N2Rs();
506 N_w_RsN_vec_t0 = [0; 0; 0];
507 [sigma_BRs_vec_t0, B_w_BRs_vec_t0] = track_err(sigma_BN_vec_t0, B_w_BN_vec_t0, RsN_t0,
508         N_w_RsN_vec_t0);
509
510 % Nadir-Pointing frame Rn
511 RnN_t0 = N2Rn(params.LMO, 0);
512 N_w_RnN_vec_t0 = inertial_w_RnN(params.LMO, 0);
513 [sigma_BRn_vec_t0, B_w_BRn_vec_t0] = track_err(sigma_BN_vec_t0, B_w_BN_vec_t0, RnN_t0,
514         N_w_RnN_vec_t0);
515
516 % GMO-Pointing frame Rc
517 RcN_t0 = N2Rc(params, 0);
518 N_w_RcN_vec_t0 = inertial_w_RcN(params, 0);
519 [sigma_BRc_vec_t0, B_w_BRc_vec_t0] = track_err(sigma_BN_vec_t0, B_w_BN_vec_t0, RcN_t0,
520         N_w_RcN_vec_t0);
521
522 Submission Formatting
523 file = fopen(fullfile(outputFolder, 'Task6_1.txt'), 'w');

```

```

521 fprintf(file, '%.5f_%.5f_%.5f', sigma_BRs_vec_t0);
522 fclose(file);
523
524 file = fopen(fullfile(outputFolder, 'Task6_2.txt'), 'w');
525 fprintf(file, '%.5f_%.5f_%.5f', B_w_BRs_vec_t0);
526 fclose(file);
527
528 file = fopen(fullfile(outputFolder, 'Task6_3.txt'), 'w');
529 fprintf(file, '%.5f_%.5f_%.5f', sigma_BRn_vec_t0);
530 fclose(file);
531
532 file = fopen(fullfile(outputFolder, 'Task6_4.txt'), 'w');
533 fprintf(file, '%.5f_%.5f_%.5f', B_w_BRn_vec_t0);
534 fclose(file);
535
536 file = fopen(fullfile(outputFolder, 'Task6_5.txt'), 'w');
537 fprintf(file, '%.5f_%.5f_%.5f', sigma_BRc_vec_t0);
538 fclose(file);
539
540 file = fopen(fullfile(outputFolder, 'Task6_6.txt'), 'w');
541 fprintf(file, '%.5f_%.5f_%.5f', B_w_BRc_vec_t0);
542 fclose(file);
543
544 Task 7: Numerical Attitude Simulator
545 Functions
546 function [X_vec_hist, u_vec_hist, t_hist] = RK4(f, u_fun, X0_vec, t_span, dt, params)
547 %=====
548 %
549 % Integrates X_dot = f(t, X, u, params) using a fixed-step RK4 scheme
550 %
551 % The structure follows the class algorithm:
552 %   1) evaluate control
553 %   2) perform one RK4 step
554 %   3) apply MRP shadow-set switch if needed
555 %   4) save state and control histories
556 %
557 % Author: G. Montseny
558 % Date: April 6, 2026
559 %
560 %
561 % INPUT:                Description                Units
562 %
563 % f                    -    dynamics function handle f(t, X_vec, u_vec, params_SC) -
564 % u_fun                -    control function handle u_fun(t, X_vec, params_ctrl) -
565 % X0_vec               -    initial state vector                -
566 % t_span               -    integration time span                s
567 % dt                   -    fixed integration time step          s
568 % params               -    structure with all parameters        -
569 %
570 % OUTPUT:
571 %
572 % X_vec_hist -         state history, one column per time step    -
573 % u_vec_hist -         control history, one column per time step  -
574 % t_hist      -         time history                                s
575 %
576 %=====
577
578 %-----
579 %-----

```

```

580 %                                     INITIALIZATION
581 %-----
582 %-----
583
584 % Make sure initial state is a column vector
585 X0_vec = X0_vec(:);
586
587 % Initial time and final time
588 t0 = t_span(1);
589 tf = t_span(2);
590
591 % Number of time steps
592 N = floor((tf - t0)/dt) + 1;
593
594 % Time history
595 t_hist = t0 + (0:N-1)*dt;
596
597 % Add final time if needed
598 if abs(t_hist(end) - tf) > 1e-12
599     t_hist = [t_hist, t_max];
600     N = length(t_hist);
601 end
602
603 % State size
604 n_X = length(X0_vec);
605
606 % Evaluate control once to get control size
607 u0_vec = u_fun(t0, X0_vec, params);
608 u0_vec = u0_vec(:);
609 n_u = length(u0_vec);
610
611 % Initialize histories
612 X_vec_hist = zeros(n_X, N);
613 u_vec_hist = zeros(n_u, N);
614
615 % Store initial values
616 X_vec_hist(:,1) = X0_vec;
617 u_vec_hist(:,1) = u0_vec;
618
619 %-----
620 %-----
621 %                                     TIME-STEPPING LOOP
622 %-----
623 %-----
624
625 for n = 1:N-1
626
627     % Current time and state
628     t_n = t_hist(n);
629     X_n_vec = X_vec_hist(:,n);
630
631     %-----
632     % Evaluate control at current step
633     % This matches the class logic where the control is updated
634     % once per integration step and then held fixed during RK4
635     %-----
636     u_n_vec = u_fun(t_n, X_n_vec, params);
637     u_n_vec = u_n_vec(:);
638     % INCLUDE MORE STUFF IN CLASS

```

```

639
640 %-----
641 % RK4 integration step
642 %-----
643 h = dt;
644 k1_vec = h * f(t_n, X_n_vec, u_n_vec, params);
645 k2_vec = h * f(t_n + h/2, X_n_vec + k1_vec/2, u_n_vec, params);
646 k3_vec = h * f(t_n + h/2, X_n_vec + k2_vec/2, u_n_vec, params);
647 k4_vec = h * f(t_n + h, X_n_vec + k3_vec, u_n_vec, params);
648
649 X_np1_vec = X_n_vec + (1/6) * (k1_vec + 2*k2_vec + 2*k3_vec + k4_vec);
650
651 %-----
652 % MRP shadow-set switching
653 % Assume the first 3 states are the MRP vector sigma_B/N
654 %-----
655 sigma_BN_vec = X_np1_vec(1:3);
656
657 if norm(sigma_BN_vec) > 1
658     X_np1_vec(1:3) = -sigma_BN_vec / norm(sigma_BN_vec)^2;
659 end
660
661 %-----
662 % Save next state and current control
663 %-----
664 X_vec_hist(:,n+1) = X_np1_vec;
665 u_vec_hist(:,n) = u_n_vec;
666
667 end
668
669 % Store final control value
670 u_vec_hist(:,N) = u_fun(t_hist(N), X_vec_hist(:,N), params);
671
672 end
673
674 function Xdot_vec = f_T7(t, X_vec, u_vec, params_mission)
675
676     params_SC = params_mission.SC;
677     % CHANGE TO VALID FOR ALL INERTIA TENSORS, NOT ONLY DIAGONAL/PRINCIPAL
678     % Column X_vec and u_vec
679     X_vec = X_vec(:);
680     u_vec = u_vec(:);
681
682     % Initialize Xdot_vec
683     Xdot_vec = zeros(size(X_vec));
684
685     % Extract values
686     B_wBN_vec = X_vec(4:6);
687     sigmaBN_vec = X_vec(1:3);
688
689     w_1 = B_wBN_vec(1); w_2 = B_wBN_vec(2); w_3 = B_wBN_vec(3);
690     u_1 = u_vec(1); u_2 = u_vec(2); u_3 = u_vec(3);
691
692     % Extract moment of inertia values (assuming B is a principal axes
693     % frame)
694     B_I_c = params_SC.B_I_c;
695     I_1 = B_I_c(1,1); I_2 = B_I_c(2,2); I_3 = B_I_c(3,3);
696
697     % Differential equations for sigma_i

```

```

698     Xdot_vec(1:3) = dMRP(sigmaBN_vec, B_wBN_vec);
699
700     % Differential equations for w_i
701     Xdot_vec(4) = (-(I_3-I_2)*w_2*w_3 + u_1)/I_1;
702     Xdot_vec(5) = (-(I_1-I_3)*w_1*w_3 + u_2)/I_2;
703     Xdot_vec(6) = (-(I_2-I_1)*w_1*w_2 + u_3)/I_3;
704
705 end
706
707
708
709 function H_vec = angularMomentum(SC, B_wBN_vec)
710     H_vec = SC.B_I_c*B_wBN_vec;
711 end
712
713
714 Code
715 u_fun = @(t, X_vec, SC) [0;0;0];
716 [X_vec_hist, u_vec_hist, t_hist] = RK4(@f_T7, u_fun, SC.X0_vec, [0,500], 1, SC);
717 t_hist(end)
718 B_wBN_vec_500 = X_vec_hist(4:6,end)
719 B_H_vec = SC.B_I_c*B_wBN_vec_500
720 T_rot_500 = 0.5*B_wBN_vec_500'*SC.B_I_c*B_wBN_vec_500
721 sigmaBN_vec_500 = X_vec_hist(1:3,end)
722 BN_500 = MRP2C(sigmaBN_vec_500);
723 N_H_vec = BN_500'*B_H_vec
724
725 u_fun = @(t, X_vec, SC) [0.01;-0.01;0.02];
726 [X_vec_hist, u_vec_hist, t_hist] = RK4(@f_T7, u_fun, SC.X0_vec, [0,100], 1, SC);
727 sigmaBN_vec_100 = X_vec_hist(1:3,end)
728
729 Submission Formatting
730 file = fopen(fullfile(outputFolder,'Task7_1.txt'),'w');
731 fprintf(file, '%.5f_%.5f_%.5f', B_H_vec);
732 fclose(file);
733
734 file = fopen(fullfile(outputFolder,'Task7_2.txt'),'w');
735 fprintf(file, '%.5f', T_rot_500);
736 fclose(file);
737
738 file = fopen(fullfile(outputFolder,'Task7_3.txt'),'w');
739 fprintf(file, '%.5f_%.5f_%.5f', sigmaBN_vec_500);
740 fclose(file);
741
742 file = fopen(fullfile(outputFolder,'Task7_4.txt'),'w');
743 fprintf(file, '%.5f_%.5f_%.5f', N_H_vec);
744 fclose(file);
745
746 file = fopen(fullfile(outputFolder,'Task7_5.txt'),'w');
747 fprintf(file, '%.5f_%.5f_%.5f', sigmaBN_vec_100);
748 fclose(file);
749
750 Task 8: Sun Pointing Control
751 Functions
752 function B_u_vec = attitude_pointing_control(t, X_vec, params_mission)
753
754     % Extract parameters
755     params_orb = params_mission.LM0;
756     params_ctrl = params_mission.ctrl;

```

```

757
758 % Extract state
759 sigma_BN_vec = X_vec(1:3);
760 B_w_BN_vec = X_vec(4:6);
761
762 % Determine reference DCM and angular velocity
763 switch params_ctrl.mode
764     case 'Sun'
765         RN = N2Rs();
766         N_w_RN_vec = [0; 0; 0];
767     case 'Nadir'
768         RN = N2Rn(params_orb, t);
769         N_w_RN_vec = inertial_w_RnN(params_orb, t);
770     case 'GMO'
771         RN = N2Rc(params_mission, t);
772         N_w_RN_vec = inertial_w_RcN(params_mission, t);
773     otherwise
774         error('Given pointing mode does not exist')
775 end
776
777 % Calculate tracking errors
778 [sigma_BR_vec, B_w_BR_vec] = track_err(sigma_BN_vec, B_w_BN_vec, RN, N_w_RN_vec);
779
780 % Extract control gains
781 P = params_ctrl.P;
782 K = params_ctrl.K;
783
784 % Simple PD Control Law
785 B_u_vec = -K*sigma_BR_vec - P*B_w_BR_vec;
786 end
787 Code
788 params_mission.ctrl.mode = 'Sun';
789 u_Sun = @(t,X_vec, params_mission) attitude_pointing_control(t,X_vec, params_mission);
790 [X_vec_hist, u_vec_hist, t_hist] = RK4(@f_T7, u_Sun, SC.X0_vec, [0,400], 1,
791     params_mission);
792 sigma_BN_vec_15 = X_vec_hist(1:3, 16);
793 sigma_BN_vec_100 = X_vec_hist(1:3, 101);
794 sigma_BN_vec_200 = X_vec_hist(1:3, 201);
795 sigma_BN_vec_400 = X_vec_hist(1:3, 401);
796 Submission
797 file = fopen(fullfile(outputFolder, 'Task8_1.txt'), 'w');
798 fprintf(file, '%.7f\n%.7f', [ctrl_PD.P; ctrl_PD.K]);
799 fclose(file);
800
801 file = fopen(fullfile(outputFolder, 'Task8_2.txt'), 'w');
802 fprintf(file, '%.7f\n%.7f\n%.7f', sigma_BN_vec_15);
803 fclose(file);
804
805 file = fopen(fullfile(outputFolder, 'Task8_3.txt'), 'w');
806 fprintf(file, '%.7f\n%.7f\n%.7f', sigma_BN_vec_100);
807 fclose(file);
808
809 file = fopen(fullfile(outputFolder, 'Task8_4.txt'), 'w');
810 fprintf(file, '%.7f\n%.7f\n%.7f', sigma_BN_vec_200);
811 fclose(file);
812
813 file = fopen(fullfile(outputFolder, 'Task8_5.txt'), 'w');
814 fprintf(file, '%.7f\n%.7f\n%.7f', sigma_BN_vec_400);
815 fclose(file);

```



```

815 SC Motion Plots
816 figure('Position', [100, 100, 900, 400]); % [left, bottom, width, height]
817 tiledlayout(2,1)
818
819 nexttile
820 plot(t_hist, X_vec_hist(1,:), 'LineWidth',1.2); hold on;
821 plot(t_hist, X_vec_hist(2,:), 'LineWidth',1.2); hold on;
822 plot(t_hist, X_vec_hist(3,:), 'LineWidth',1.2); hold on;
823 %plot(t_hist, vecnorm(X_vec_hist(1:3,:))); hold on;
824 grid on;
825 ylabel('$\sigma_{\{B/N,\;i\}}$'); %xlabel('$t$ (s)');
826 legend('$\sigma_{\{B/N,1\}}$', '$\sigma_{\{B/N,2\}}$', '$\sigma_{\{B/N,3\}}$', 'Location', '
    eastoutside');
827
828 nexttile
829 plot(t_hist, X_vec_hist(4,:), 'LineWidth',1.2); hold on;
830 plot(t_hist, X_vec_hist(5,:), 'LineWidth',1.2); hold on;
831 plot(t_hist, X_vec_hist(6,:), 'LineWidth',1.2); hold on;
832 grid on;
833 ylabel('$^B\omega_{\{B/N,\;i\}}$ (rad/s)'); xlabel('$t$ (s)');
834 legend('$^B\omega_{\{B/N,1\}}$', '$^B\omega_{\{B/N,2\}}$', '$^B\omega_{\{B/N,3\}}$', '
    Location', 'eastoutside');
835 exportgraphics(gcf, 'Sun_1.pdf', 'ContentType','vector');
836 Tracking Performance & Control
837 % CALCULATE
838 sigma_BN_vec_hist = X_vec_hist(1:3, :);
839 N = size(sigma_BN_vec_hist,2);
840 sigma_BR_vec_hist = zeros(3,N);
841 RsN = N2Rs();
842 for k = 1:N
843     % Extract sigma_BN at time k
844     sigma_BN_vec = sigma_BN_vec_hist(:,k);
845
846     % Convert to DCM
847     BN = MRP2C(sigma_BN_vec);
848
849     % Compose attitude
850     BR = BN * RsN';
851
852     % Convert back to MRP
853     sigma_BR_vec = C2MRP(BR);
854
855     % Store
856     sigma_BR_vec_hist(:,k) = sigma_BR_vec;
857 end
858
859 B_w_BR_vec_hist = X_vec_hist(4:6, :);
860
861 % Plot
862 figure('Position', [100, 100, 900, 600]); % [left, bottom, width, height]
863 tiledlayout(3,1)
864
865 nexttile
866 plot(t_hist, sigma_BR_vec_hist(1,:), 'LineWidth',1.2); hold on;
867 plot(t_hist, sigma_BR_vec_hist(2,:), 'LineWidth',1.2); hold on;
868 plot(t_hist, sigma_BR_vec_hist(3,:), 'LineWidth',1.2); hold on;
869 %plot(t_hist, vecnorm(X_vec_hist(1:3,:))); hold on;
870 grid on;

```

```

872 ylabel('$\sigma_{\{B/R_s, \; i\}}$'); %xlabel('$t$ (s)');
873 legend('$\sigma_{\{B/R_s, 1\}}$', '$\sigma_{\{B/R_s, 2\}}$', '$\sigma_{\{B/R_s, 3\}}$', '
      Location', 'eastoutside');
874
875 nexttile
876 plot(t_hist, B_w_BR_vec_hist(1,:), 'LineWidth',1.2); hold on;
877 plot(t_hist, B_w_BR_vec_hist(2,:), 'LineWidth',1.2); hold on;
878 plot(t_hist, B_w_BR_vec_hist(3,:), 'LineWidth',1.2); hold on;
879 grid on;
880 ylabel('$^B\omega_{\{B/R_s, \; i\}}_{\perp}(\text{rad/s})$'); %xlabel('$t$ (s)');
881 legend('$^B\omega_{\{B/R_s, 1\}}$', '$^B\omega_{\{B/R_s, 2\}}$', '$^B\omega_{\{B/R_s, 3\}}$', '
      Location', 'eastoutside');
882
883
884 nexttile
885 plot(t_hist, u_vec_hist(1,:), 'LineWidth',1.2); hold on;
886 plot(t_hist, u_vec_hist(2,:), 'LineWidth',1.2); hold on;
887 plot(t_hist, u_vec_hist(3,:), 'LineWidth',1.2); hold on;
888 grid on;
889 ylabel('$^B u_{i\perp}(\text{kgm}^2\text{s}^{-2})$'); xlabel('$t_{\perp}(\text{s})$');
890 legend('$^B u_{1\perp}$', '$^B u_{2\perp}$', '$^B u_{3\perp}$', 'Location', 'eastoutside');
891 exportgraphics(gcf, 'Sun_2.pdf', 'ContentType','vector');
892
893 Task 9: Nadir Pointing Control
894 Code
895 params_mission.ctrl.mode = 'Nadir';
896 u_Nadir = @(t,X_vec, params_mission) attitude_pointing_control(t,X_vec, params_mission
);
897 [X_vec_hist, u_vec_hist, t_hist] = RK4(@f_T7, u_Nadir, SC.X0_vec, [0,400], 1,
      params_mission);
898 sigma_BN_vec_15 = X_vec_hist(1:3, 16);
899 sigma_BN_vec_100 = X_vec_hist(1:3, 101);
900 sigma_BN_vec_200 = X_vec_hist(1:3, 201);
901 sigma_BN_vec_400 = X_vec_hist(1:3, 401);
902 Submission
903 file = fopen(fullfile(outputFolder,'Task9_1.txt'),'w');
904 fprintf(file, '%.7f_%.7f_%.7f', sigma_BN_vec_15);
905 fclose(file);
906
907 file = fopen(fullfile(outputFolder,'Task9_2.txt'),'w');
908 fprintf(file, '%.7f_%.7f_%.7f', sigma_BN_vec_100);
909 fclose(file);
910
911 file = fopen(fullfile(outputFolder,'Task9_3.txt'),'w');
912 fprintf(file, '%.7f_%.7f_%.7f', sigma_BN_vec_200);
913 fclose(file);
914
915 file = fopen(fullfile(outputFolder,'Task9_4.txt'),'w');
916 fprintf(file, '%.7f_%.7f_%.7f', sigma_BN_vec_400);
917 fclose(file);
918 SC Motion Plots
919 figure('Position', [100, 100, 900, 400]); % [left, bottom, width, height]
920 tiledlayout(2,1)
921
922 nexttile
923 plot(t_hist, X_vec_hist(1,:), 'LineWidth',1.2); hold on;
924 plot(t_hist, X_vec_hist(2,:), 'LineWidth',1.2); hold on;
925 plot(t_hist, X_vec_hist(3,:), 'LineWidth',1.2); hold on;
926 %plot(t_hist, vecnorm(X_vec_hist(1:3,:))); hold on;

```

```

927 grid on;
928 ylabel('$\sigma_{\{B/N,\;i\}}$'); %xlabel('$t$ (s)');
929 legend('$\sigma_{\{B/N,1\}}$', '$\sigma_{\{B/N,2\}}$', '$\sigma_{\{B/N,3\}}$', 'Location', '
    eastoutside');
930
931 nexttile
932 plot(t_hist, X_vec_hist(4,:), 'LineWidth',1.2); hold on;
933 plot(t_hist, X_vec_hist(5,:), 'LineWidth',1.2); hold on;
934 plot(t_hist, X_vec_hist(6,:), 'LineWidth',1.2); hold on;
935 grid on;
936 ylabel('$^B\omega_{\{B/N,\;i\}}$ (rad/s)'); xlabel('$t$ (s)');
937 legend('$^B\omega_{\{B/N,1\}}$', '$^B\omega_{\{B/N,2\}}$', '$^B\omega_{\{B/N,3\}}$', '
    Location', 'eastoutside');
938 exportgraphics(gcf, 'Nadir_1.pdf', 'ContentType','vector');
939 Tracking Performance & Control
940 % CALCULATE
941 sigma_BN_vec_hist = X_vec_hist(1:3, :);
942 B_w_BN_vec_hist = X_vec_hist(4:6, :);
943 N = size(sigma_BN_vec_hist,2);
944 sigma_BR_vec_hist = zeros(3,N);
945 B_w_BR_vec_hist = zeros(3,N);
946
947
948 for k = 1:N
949     % Extract sigma_BN and w_BN at time k
950     sigma_BN_vec = sigma_BN_vec_hist(:,k);
951     B_w_BN_vec = B_w_BN_vec_hist(:,k);
952
953     % Extract time k
954     t_k = t_hist(k);
955
956     % Extract RnN
957     RnN = N2Rn(LM0, t_k);
958
959     % Extract w_RnN
960     N_w_RnN_vec = inertial_w_RnN(LM0, t_k);
961
962     % Calculate track errors
963     [sigma_BR_vec, B_w_BR_vec] = track_err(sigma_BN_vec, B_w_BN_vec, RnN, N_w_RnN_vec)
964     ;
965
966     % Store
967     sigma_BR_vec_hist(:,k) = sigma_BR_vec;
968     B_w_BR_vec_hist(:,k) = B_w_BR_vec;
969
970 end
971
972
973 % Plot
974 figure('Position', [100, 100, 900, 600]); % [left, bottom, width, height]
975 tiledlayout(3,1)
976
977 nexttile
978 plot(t_hist, sigma_BR_vec_hist(1,:), 'LineWidth',1.2); hold on;
979 plot(t_hist, sigma_BR_vec_hist(2,:), 'LineWidth',1.2); hold on;
980 plot(t_hist, sigma_BR_vec_hist(3,:), 'LineWidth',1.2); hold on;
981 %plot(t_hist, vecnorm(X_vec_hist(1:3,:))); hold on;
982

```

```

983 grid on;
984 ylabel('$\sigma_{\{B/R_n, \; i\}}$'); xlabel('$t$ (s)');
985 legend('$\sigma_{\{B/R_n, 1\}}$', '$\sigma_{\{B/R_n, 2\}}$', '$\sigma_{\{B/R_n, 3\}}$', '
    Location', 'eastoutside');
986
987 nexttile
988 plot(t_hist, B_w_BR_vec_hist(1,:), 'LineWidth',1.2); hold on;
989 plot(t_hist, B_w_BR_vec_hist(2,:), 'LineWidth',1.2); hold on;
990 plot(t_hist, B_w_BR_vec_hist(3,:), 'LineWidth',1.2); hold on;
991 grid on;
992 ylabel('$^B\omega_{\{B/R_n, \; i\}}$ (rad/s)'); xlabel('$t$ (s)');
993 legend('$^B\omega_{\{B/R_n, 1\}}$', '$^B\omega_{\{B/R_n, 2\}}$', '$^B\omega_{\{B/R_n, 3\}}$', '
    Location', 'eastoutside');
994
995
996 nexttile
997 plot(t_hist, u_vec_hist(1,:), 'LineWidth',1.2); hold on;
998 plot(t_hist, u_vec_hist(2,:), 'LineWidth',1.2); hold on;
999 plot(t_hist, u_vec_hist(3,:), 'LineWidth',1.2); hold on;
1000 grid on;
1001 ylabel('$^Bu_{\downarrow}$ (kgm$^2s^{-2}$)'); xlabel('$t$ (s)');
1002 legend('$^Bu_1$', '$^Bu_2$', '$^Bu_3$', 'Location', 'eastoutside');
1003 exportgraphics(gcf, 'Nadir_2.pdf', 'ContentType','vector');
1004
1005 Task 10: GMO Pointing Control
1006 Code
1007 params_mission.ctrl.mode = 'GMO';
1008 u_GMO = @(t,X_vec, params_mission) attitude_pointing_control(t,X_vec, params_mission);
1009 [X_vec_hist, u_vec_hist, t_hist] = RK4(@f_T7, u_GMO, SC.X0_vec, [0,400], 1,
    params_mission);
1010 sigma_BN_vec_15 = X_vec_hist(1:3, 16);
1011 sigma_BN_vec_100 = X_vec_hist(1:3, 101);
1012 sigma_BN_vec_200 = X_vec_hist(1:3, 201);
1013 sigma_BN_vec_400 = X_vec_hist(1:3, 401);
1014 Submission
1015 file = fopen(fullfile(outputFolder, 'Task10_1.txt'), 'w');
1016 fprintf(file, '%.7f_%.7f_%.7f', sigma_BN_vec_15);
1017 fclose(file);
1018
1019 file = fopen(fullfile(outputFolder, 'Task10_2.txt'), 'w');
1020 fprintf(file, '%.7f_%.7f_%.7f', sigma_BN_vec_100);
1021 fclose(file);
1022
1023 file = fopen(fullfile(outputFolder, 'Task10_3.txt'), 'w');
1024 fprintf(file, '%.7f_%.7f_%.7f', sigma_BN_vec_200);
1025 fclose(file);
1026
1027 file = fopen(fullfile(outputFolder, 'Task10_4.txt'), 'w');
1028 fprintf(file, '%.7f_%.7f_%.7f', sigma_BN_vec_400);
1029 fclose(file);
1030 SC Motion Plots
1031 figure('Position', [100, 100, 900, 400]); % [left, bottom, width, height]
1032 tiledlayout(2,1)
1033
1034 nexttile
1035 plot(t_hist, X_vec_hist(1,:), 'LineWidth',1.2); hold on;
1036 plot(t_hist, X_vec_hist(2,:), 'LineWidth',1.2); hold on;
1037 plot(t_hist, X_vec_hist(3,:), 'LineWidth',1.2); hold on;
1038 %plot(t_hist, vecnorm(X_vec_hist(1:3,:))); hold on;

```

```

1039 grid on;
1040 ylabel('$\sigma_{\{B/N,\;i\}}$'); %xlabel('$t$ (s)');
1041 legend('$\sigma_{\{B/N,1\}}$', '$\sigma_{\{B/N,2\}}$', '$\sigma_{\{B/N,3\}}$', 'Location', '
    eastoutside');
1042
1043 nexttile
1044 plot(t_hist, X_vec_hist(4,:), 'LineWidth',1.2); hold on;
1045 plot(t_hist, X_vec_hist(5,:), 'LineWidth',1.2); hold on;
1046 plot(t_hist, X_vec_hist(6,:), 'LineWidth',1.2); hold on;
1047 grid on;
1048 ylabel('$^B\omega_{\{B/N,\;i\}}$ (rad/s)'); xlabel('$t$ (s)');
1049 legend('$^B\omega_{\{B/N,1\}}$', '$^B\omega_{\{B/N,2\}}$', '$^B\omega_{\{B/N,3\}}$', '
    Location', 'eastoutside');
1050 exportgraphics(gcf, 'GMO_1.pdf', 'ContentType','vector');
1051 Tracking Performance & Control
1052 % CALCULATE
1053 sigma_BN_vec_hist = X_vec_hist(1:3, :);
1054 B_w_BN_vec_hist = X_vec_hist(4:6, :);
1055 N = size(sigma_BN_vec_hist,2);
1056 sigma_BR_vec_hist = zeros(3,N);
1057 B_w_BR_vec_hist = zeros(3,N);
1058
1059
1060 for k = 1:N
1061     % Extract sigma_BN and w_BN at time k
1062     sigma_BN_vec = sigma_BN_vec_hist(:,k);
1063     B_w_BN_vec = B_w_BN_vec_hist(:,k);
1064
1065     % Extract time k
1066     t_k = t_hist(k);
1067
1068     % Extract RcN
1069     RcN = N2Rc(params_mission, t_k);
1070
1071     % Extract w_RcN
1072     N_w_RcN_vec = inertial_w_RcN(params_mission, t_k);
1073
1074     % Calculate track errors
1075     [sigma_BR_vec, B_w_BR_vec] = track_err(sigma_BN_vec, B_w_BN_vec, RcN, N_w_RcN_vec)
        ;
1076
1077     % Store
1078     sigma_BR_vec_hist(:,k) = sigma_BR_vec;
1079     B_w_BR_vec_hist(:,k) = B_w_BR_vec;
1080
1081 end
1082
1083
1084
1085 % Plot
1086 figure('Position', [100, 100, 900, 600]); % [left, bottom, width, height]
1087 tiledlayout(3,1)
1088
1089 nexttile
1090 plot(t_hist, sigma_BR_vec_hist(1,:), 'LineWidth',1.2); hold on;
1091 plot(t_hist, sigma_BR_vec_hist(2,:), 'LineWidth',1.2); hold on;
1092 plot(t_hist, sigma_BR_vec_hist(3,:), 'LineWidth',1.2); hold on;
1093 %plot(t_hist, vecnorm(X_vec_hist(1:3,:))); hold on;
1094 grid on;

```

```

1095 ylabel('$\sigma_{\{B/R_c, \; i\}}$'); %xlabel('$t$ (s)');
1096 legend('$\sigma_{\{B/R_c, 1\}}$', '$\sigma_{\{B/R_c, 2\}}$', '$\sigma_{\{B/R_c, 3\}}$', '
      Location', 'eastoutside');
1097
1098 nexttile
1099 plot(t_hist, B_w_BR_vec_hist(1,:), 'LineWidth',1.2); hold on;
1100 plot(t_hist, B_w_BR_vec_hist(2,:), 'LineWidth',1.2); hold on;
1101 plot(t_hist, B_w_BR_vec_hist(3,:), 'LineWidth',1.2); hold on;
1102 grid on;
1103 ylabel('$^B\omega_{\{B/R_c, \; i\}}_{\perp}(\text{rad/s})$'); %xlabel('$t$ (s)');
1104 legend('$^B\omega_{\{B/R_c, 1\}}$', '$^B\omega_{\{B/R_c, 2\}}$', '$^B\omega_{\{B/R_c, 3\}}$', '
      Location', 'eastoutside');
1105
1106
1107 nexttile
1108 plot(t_hist, u_vec_hist(1,:), 'LineWidth',1.2); hold on;
1109 plot(t_hist, u_vec_hist(2,:), 'LineWidth',1.2); hold on;
1110 plot(t_hist, u_vec_hist(3,:), 'LineWidth',1.2); hold on;
1111 grid on;
1112 ylabel('$^B u_{i\perp}(\text{kgm}^2\text{s}^{-2})$'); xlabel('$t_{\perp}(\text{s})$');
1113 legend('$^B u_{1\perp}$', '$^B u_{2\perp}$', '$^B u_{3\perp}$', 'Location', 'eastoutside');
1114 exportgraphics(gcf, 'GMO_2.pdf', 'ContentType','vector');
1115
1116 Task 11: Mission Scenario Simulation
1117 Functions
1118 function [X_vec_hist, u_vec_hist, t_hist] = integrate_mission(f, u_fun, X0_vec, t_span
      , dt, params)
1119 %=====
1120 %
1121 % Integrates X_dot = f(t, X, u, params) using a fixed-step RK4 scheme
1122 %
1123 % The structure follows the class algorithm:
1124 % 1) evaluate control
1125 % 2) perform one RK4 step
1126 % 3) apply MRP shadow-set switch if needed
1127 % 4) save state and control histories
1128 % SOLVE PARAMS INPUT: SHOULD ID EVEN BE ONE?
1129 %
1130 % Author: G. Montseny
1131 % Date: April 6, 2026
1132 %
1133 %
1134 % INPUT:          Description          Units
1135 %
1136 % f              -   dynamics function handle f(t, X_vec, u_vec, params_SC) -
1137 % u_fun          -   control function handle u_fun(t, X_vec, params_ctrl)   -
1138 % X0_vec         -   initial state vector                                  -
1139 % t_span         -   integration time span                                s
1140 % dt             -   fixed integration time step                          s
1141 % params         -   structure with all parameters                        -
1142 %
1143 % OUTPUT:
1144 %
1145 % X_vec_hist     -   state history, one column per time step              -
1146 % u_vec_hist     -   control history, one column per time step             -
1147 % t_hist         -   time history                                           s
1148 %
1149 %=====
1150

```

```

1151 %-----
1152 %-----
1153 %                               INITIALIZATION
1154 %-----
1155 %-----
1156
1157 % Make sure initial state is a column vector
1158 X0_vec = X0_vec(:);
1159
1160 % Initial time and final time
1161 t0 = t_span(1);
1162 tf = t_span(2);
1163
1164 % Number of time steps
1165 N = floor((tf - t0)/dt) + 1;
1166
1167 % Time history
1168 t_hist = t0 + (0:N-1)*dt;
1169
1170 % Add final time if needed
1171 if abs(t_hist(end) - tf) > 1e-12
1172     t_hist = [t_hist, t_max];
1173     N = length(t_hist);
1174 end
1175
1176 % State size
1177 n_X = length(X0_vec);
1178
1179 % Evaluate control once to get control size
1180 u0_vec = u_fun(t0, X0_vec, params);
1181 u0_vec = u0_vec(:);
1182 n_u = length(u0_vec);
1183
1184 % Initialize histories
1185 X_vec_hist = zeros(n_X, N);
1186 u_vec_hist = zeros(n_u, N);
1187
1188 % Store initial values
1189 X_vec_hist(:,1) = X0_vec;
1190 u_vec_hist(:,1) = u0_vec;
1191
1192 %-----
1193 %-----
1194 %                               TIME-STEPPING LOOP
1195 %-----
1196 %-----
1197
1198 for n = 1:N-1
1199
1200     % Current time and state
1201     t_n = t_hist(n);
1202     X_n_vec = X_vec_hist(:,n);
1203
1204     %-----
1205     % Evaluate control at current step
1206     % This matches the class logic where the control is updated
1207     % once per integration step and then held fixed during RK4
1208     %-----
1209

```

```

1210 % Extract LMO and GMO posiitons
1211 [N_rLMO_vec, ~] = inertial_state(params.LMO, t_n); r_LMO = norm(N_rLMO_vec);
1212 [N_rGMO_vec, ~] = inertial_state(params.GMO, t_n); r_GMO = norm(N_rGMO_vec);
1213
1214
1215 % Conditions
1216 if dot(N_rLMO_vec, [0;1;0]) > 0 % SUN
1217     params.ctrl.mode = 'Sun';
1218 else
1219     gamma = acosd(N_rLMO_vec'*N_rGMO_vec/(r_LMO*r_GMO));
1220
1221     if gamma >= 35 % NADIR
1222         params.ctrl.mode = 'Nadir';
1223
1224     else % GMO
1225         params.ctrl.mode = 'GMO';
1226
1227     end
1228
1229 end
1230
1231 % Control vector
1232 u_n_vec = u_fun(t_n, X_n_vec, params);
1233 u_n_vec = u_n_vec(:);
1234
1235 %-----
1236 % RK4 integration step
1237 %-----
1238 h = dt;
1239 k1_vec = h * f(t_n, X_n_vec, u_n_vec, params);
1240 k2_vec = h * f(t_n + h/2, X_n_vec + k1_vec/2, u_n_vec, params);
1241 k3_vec = h * f(t_n + h/2, X_n_vec + k2_vec/2, u_n_vec, params);
1242 k4_vec = h * f(t_n + h, X_n_vec + k3_vec, u_n_vec, params);
1243
1244 X_np1_vec = X_n_vec + (1/6) * (k1_vec + 2*k2_vec + 2*k3_vec + k4_vec);
1245
1246 %-----
1247 % MRP shadow-set switching
1248 % Assume the first 3 states are the MRP vector sigma_B/N
1249 %-----
1250 sigma_BN_vec = X_np1_vec(1:3);
1251
1252 if norm(sigma_BN_vec) > 1
1253     X_np1_vec(1:3) = -sigma_BN_vec / norm(sigma_BN_vec)^2;
1254 end
1255
1256 %-----
1257 % Save next state and current control
1258 %-----
1259 X_vec_hist(:,n+1) = X_np1_vec;
1260 u_vec_hist(:,n) = u_n_vec;
1261
1262 end
1263
1264 % Store final control value
1265 u_vec_hist(:,N) = u_fun(t_hist(N), X_vec_hist(:,N), params);
1266
1267 end
1268 Code

```



```

1269 u_mission = @(t,X_vec, params_mission) attitude_pointing_control(t,X_vec,
      params_mission);
1270 [X_vec_hist, u_vec_hist, t_hist] = integrate_mission(@f_T7, u_mission, SC.X0_vec,
      [0,6500], 1, params_mission);
1271 sigma_BN_vec_300 = X_vec_hist(1:3, 301);
1272 sigma_BN_vec_2100 = X_vec_hist(1:3, 2101);
1273 sigma_BN_vec_3400 = X_vec_hist(1:3, 3401);
1274 sigma_BN_vec_4400 = X_vec_hist(1:3, 4401);
1275 sigma_BN_vec_5600 = X_vec_hist(1:3, 5601);
1276 Submission
1277 file = fopen(fullfile(outputFolder,'Task11_1.txt'),'w');
1278 fprintf(file, '%.7f_%.7f_%.7f', sigma_BN_vec_300);
1279 fclose(file);
1280
1281 file = fopen(fullfile(outputFolder,'Task11_2.txt'),'w');
1282 fprintf(file, '%.7f_%.7f_%.7f', sigma_BN_vec_2100);
1283 fclose(file);
1284
1285 file = fopen(fullfile(outputFolder,'Task11_3.txt'),'w');
1286 fprintf(file, '%.7f_%.7f_%.7f', sigma_BN_vec_3400);
1287 fclose(file);
1288
1289 file = fopen(fullfile(outputFolder,'Task11_4.txt'),'w');
1290 fprintf(file, '%.7f_%.7f_%.7f', sigma_BN_vec_4400);
1291 fclose(file);
1292
1293 file = fopen(fullfile(outputFolder,'Task11_5.txt'),'w');
1294 fprintf(file, '%.7f_%.7f_%.7f', sigma_BN_vec_5600);
1295 fclose(file);
1296
1297 Mission Profile
1298 figure('Position', [100, 100, 900, 300]);
1299 hold on
1300 grid on
1301 box on
1302
1303 xlim([t_hist(1) t_hist(end)])
1304
1305 %-----
1306 % Left axis: r_LMO2
1307 %-----
1308 yyaxis left
1309 ylabel('$^N r_{LMO,2}$ (km)', 'Interpreter', 'latex')
1310 ylim([min(r_LMO2_hist) max(r_LMO2_hist)])
1311 yl_left = ylim;
1312
1313 % Colors
1314 c_sun = [1.00 0.90 0.75];
1315 c_sci = [0.82 0.92 0.82];
1316 c_comm = [0.90 0.90 0.90];
1317
1318 % Find switching indices
1319 idx_change = [1, find(diff(mode_hist)~=0)+1, length(t_hist)];
1320
1321 %-----
1322 % Colored intervals + labels
1323 %-----
1324 for j = 1:length(idx_change)-1
1325     i1 = idx_change(j);

```

```

1326     i2 = idx_change(j+1);
1327
1328     t1 = t_hist(i1);
1329     t2 = t_hist(i2);
1330
1331     if mode_hist(i1) == 1
1332         c = c_sun;    label_txt = 'Sun';
1333     elseif mode_hist(i1) == 2
1334         c = c_sci;    label_txt = 'Nadir';
1335     else
1336         c = c_comm;   label_txt = 'GMO';
1337     end
1338
1339     patch([t1 t2 t2 t1], [yl_left(1) yl_left(1) yl_left(2) yl_left(2)], ...
1340           c, 'EdgeColor','none', 'FaceAlpha',0.45);
1341
1342     % Label position (cleaner placement)
1343     if (t2 - t1) > 250
1344         t_mid = t1 + 0.5*(t2 - t1);    % slight right bias (fixes first Sun issue)
1345         y_txt = yl_left(1) + 0.5*(yl_left(2)-yl_left(1));
1346
1347         text(t_mid, y_txt, label_txt, ...
1348              'HorizontalAlignment','center', ...
1349              'FontSize',10, ...
1350              'FontWeight','bold', ...
1351              'Color',[0.2 0.2 0.2]);
1352     end
1353 end
1354
1355 for j = 2:length(idx_change)-1
1356     t_sw = t_hist(idx_change(j))
1357     hLine = xline(t_sw, '--', 'Color', [0.35 0.35 0.35], 'LineWidth', 0.9);
1358     hLine.Annotation.LegendInformation.IconDisplayStyle = 'off';
1359 end
1360
1361 % Send patches to back
1362 uistack(findobj(gca,'Type','patch'),'bottom')
1363
1364 %-----
1365 % Plot signals
1366 %-----
1367 yyaxis left
1368 h1 = plot(t_hist, r_LMO2_hist, '-', 'LineWidth',2, 'Color','k');
1369 ax = gca;
1370 ax.YColor = 'k';
1371
1372 yyaxis right
1373 h2 = plot(t_hist, gamma_hist, '-.', 'LineWidth',2, 'Color','k');
1374 ylabel('$\gamma_{LMO,2}(deg)', 'Interpreter','latex')
1375 ax.YColor = 'k';
1376
1377 xlabel('$t(s)', 'Interpreter','latex')
1378
1379 %-----
1380 % Legend (non-intrusive placement)
1381 %-----
1382 legend([h1 h2], ...
1383        {'$N_{LMO,2}$', '$\gamma$'}, ...
1384        'Interpreter','latex', ...

```

```

1385     'Location','southwest', ... % <- key change
1386     'Box','off');
1387 exportgraphics(gcf, 'Mission_0.pdf', 'ContentType','vector');
1388 function add_mode_shading(t_hist, mode_hist, yl)
1389
1390     hold on
1391
1392     c_sun = [1.00 0.90 0.75];
1393     c_sci = [0.82 0.92 0.82];
1394     c_comm = [0.90 0.90 0.90];
1395
1396     idx_change = [1, find(diff(mode_hist)~=0)+1, length(t_hist)];
1397
1398     for j = 1:length(idx_change)-1
1399         i1 = idx_change(j);
1400         i2 = idx_change(j+1);
1401
1402         t1 = t_hist(i1);
1403         t2 = t_hist(i2);
1404
1405         if mode_hist(i1) == 1
1406             c = c_sun;
1407         elseif mode_hist(i1) == 2
1408             c = c_sci;
1409         else
1410             c = c_comm;
1411         end
1412
1413         hPatch = patch([t1 t2 t2 t1], [yl(1) yl(1) yl(2) yl(2)], ...
1414             c, 'EdgeColor','none', 'FaceAlpha',0.30);
1415         hPatch.Annotation.LegendInformation.IconDisplayStyle = 'off';
1416     end
1417
1418     for j = 2:length(idx_change)-1
1419         t_sw = t_hist(idx_change(j));
1420         hLine = xline(t_sw, '--', 'Color', [0.35 0.35 0.35], 'LineWidth', 0.9);
1421         hLine.Annotation.LegendInformation.IconDisplayStyle = 'off';
1422     end
1423
1424     uistack(findobj(gca,'Type','patch'),'bottom')
1425 end
1426 SC Motion Plots
1427 figure('Position', [100, 100, 900, 1200]); % [left, bottom, width, height]
1428 tiledlayout(2,1)
1429
1430 nexttile
1431 plot(t_hist, X_vec_hist(1,:), 'LineWidth', 1.2); hold on;
1432 plot(t_hist, X_vec_hist(2,:), 'LineWidth', 1.2); hold on;
1433 plot(t_hist, X_vec_hist(3,:), 'LineWidth', 1.2); hold on;
1434 xlim([t_hist(1) t_hist(end)])
1435 add_mode_shading(t_hist, mode_hist, [-1,1])
1436 grid on;
1437 ylabel('$\sigma_{\{B/N,i\}}$'); %xlabel('$t$ (s)');
1438 legend('$\sigma_{\{B/N,1\}}$', '$\sigma_{\{B/N,2\}}$', '$\sigma_{\{B/N,3\}}$', 'Location', '
    eastoutside');
1439
1440 nexttile
1441 plot(t_hist, X_vec_hist(4,:), 'LineWidth', 1.2); hold on;
1442 plot(t_hist, X_vec_hist(5,:), 'LineWidth', 1.2); hold on;

```

```

1443 plot(t_hist, X_vec_hist(6,:), 'LineWidth', 1.2); hold on;
1444 xlim([t_hist(1) t_hist(end)])
1445 grid on;
1446 ylabel('$^B\omega_{\{B/N,\;i\}}$(rad/s)'); xlabel('$t$(s)');
1447 legend('$^B\omega_{\{B/N,1\}}$', '$^B\omega_{\{B/N,2\}}$', '$^B\omega_{\{B/N,3\}}$', '
    Location', 'eastoutside');
1448 add_mode_shading(t_hist, mode_hist, [-0.04,0.04])
1449 exportgraphics(gcf, 'Mission_1.pdf', 'ContentType','vector');
1450
1451 Tracking Performance & Control
1452 function [sigma_BR_vec_hist, B_w_BR_vec_hist, gamma_hist, r_LMO2_hist, mode_hist] =
    mission_analysis_hist(sigma_BN_vec_hist, B_w_BN_vec_hist, t_hist, params)
1453 %=====
1454 %
1455 % Computes mission-related post-processed histories from integrated state
1456 % histories.
1457 %
1458 % For each time step, this function computes:
1459 % 1) attitude tracking error sigma_B/R
1460 % 2) angular velocity tracking error B_w_B/R
1461 % 3) gamma angle between LMO and GMO position vectors
1462 % 4) second inertial component of LMO position
1463 %
1464 % Author: G. Montseny
1465 % Date: April 20, 2026
1466 %
1467 %
1468 % INPUT:                Description                Units
1469 %
1470 % sigma_BN_vec_hist    -    MRP history B wrt N (3 x N)    -
1471 % B_w_BN_vec_hist      -    angular velocity history B wrt N (3 x N) rad/s
1472 % t_hist               -    time history (1 x N or N x 1)    s
1473 % params               -    mission/orbit parameter struct    -
1474 %
1475 % OUTPUT:
1476 %
1477 % sigma_BR_vec_hist    -    MRP tracking error history (3 x N)    -
1478 % B_w_BR_vec_hist      -    angular velocity error history (3 x N) rad/s
1479 % gamma_hist           -    gamma angle history (1 x N)        deg
1480 % r_LMO2_hist          -    2nd inertial component history of LMO    -
1481 %
1482 %=====
1483
1484 % Ensure time is row vector
1485 t_hist = t_hist(:).';
1486 N = length(t_hist);
1487
1488 % Preallocate
1489 sigma_BR_vec_hist = zeros(3, N);
1490 B_w_BR_vec_hist   = zeros(3, N);
1491 gamma_hist        = zeros(1, N);
1492 r_LMO2_hist       = zeros(1, N);
1493 mode_hist         = zeros(1, N);
1494
1495 for k = 1:N
1496
1497     %-----
1498     % Extract current state
1499     %-----

```

```

1500     sigma_BN_vec = sigma_BN_vec_hist(:,k);
1501     B_w_BN_vec   = B_w_BN_vec_hist(:,k);
1502     t_k          = t_hist(k);
1503
1504     %-----
1505     % Inertial geometry
1506     %-----
1507     [N_rLMO_vec, ~] = inertial_state(params.LMO, t_k);
1508     [N_rGMO_vec, ~] = inertial_state(params.GMO, t_k);
1509
1510     r_LMO = norm(N_rLMO_vec);
1511     r_GMO = norm(N_rGMO_vec);
1512
1513     gamma_hist(k) = acosd(N_rLMO_vec' * N_rGMO_vec / (r_LMO * r_GMO));
1514     r_LMO2_hist(k) = N_rLMO_vec(2);
1515
1516     %-----
1517     % Determine reference frame from mission logic
1518     %-----
1519     if dot(N_rLMO_vec, [0;1;0]) > 0
1520         RN = N2Rs();
1521         N_w_RN_vec = [0;0;0];
1522         mode_hist(k) = 1; % Sun
1523
1524     else
1525         gamma = gamma_hist(k);
1526
1527         if gamma >= 35
1528             RN = N2Rn(params.LMO, t_k);
1529             N_w_RN_vec = inertial_w_RnN(params.LMO, t_k);
1530             mode_hist(k) = 2; % Science
1531         else
1532             RN = N2Rc(params, t_k);
1533             N_w_RN_vec = inertial_w_RcN(params, t_k);
1534             mode_hist(k) = 3; % Communication
1535         end
1536     end
1537
1538     %-----
1539     % Compute tracking errors
1540     %-----
1541     [sigma_BR_vec, B_w_BR_vec] = track_err(sigma_BN_vec, B_w_BN_vec, RN,
1542         N_w_RN_vec);
1543
1544     % Store
1545     sigma_BR_vec_hist(:,k) = sigma_BR_vec;
1546     B_w_BR_vec_hist(:,k)   = B_w_BR_vec;
1547 end
1548
1549 [sigma_BR_vec_hist, B_w_BR_vec_hist, gamma_hist, r_LMO2_hist, mode_hist] =
1550 mission_analysis_hist(X_vec_hist(1:3,:), X_vec_hist(4:6,:), t_hist, params_mission)
1551 ;
1552 figure('Position', [100, 100, 900, 1200]); % [left, bottom, width, height]
1553 tiledlayout(3,1)
1554
1555 nexttile

```

```

1556 plot(t_hist, sigma_BR_vec_hist(1,:), 'LineWidth',1.2); hold on;
1557 plot(t_hist, sigma_BR_vec_hist(2,:), 'LineWidth',1.2); hold on;
1558 plot(t_hist, sigma_BR_vec_hist(3,:), 'LineWidth',1.2); hold on;
1559 xlim([t_hist(1) t_hist(end)])
1560 %plot(t_hist, vecnorm(X_vec_hist(1:3,:))); hold on;
1561 add_mode_shading(t_hist, mode_hist, [-1,1])
1562 grid on;
1563 ylabel('$\sigma_{\{B/R_c,\;i\}}$'); %xlabel('$t$ (s)');
1564 legend('$\sigma_{\{B/R_c,1\}}$', '$\sigma_{\{B/R_c,2\}}$', '$\sigma_{\{B/R_c,3\}}$', '
    Location', 'eastoutside');
1565
1566 nexttile
1567 plot(t_hist, B_w_BR_vec_hist(1,:), 'LineWidth',1.2); hold on;
1568 plot(t_hist, B_w_BR_vec_hist(2,:), 'LineWidth',1.2); hold on;
1569 plot(t_hist, B_w_BR_vec_hist(3,:), 'LineWidth',1.2); hold on;
1570 xlim([t_hist(1) t_hist(end)])
1571 add_mode_shading(t_hist, mode_hist, [-0.04,0.04])
1572 grid on;
1573 ylabel('$^B\omega_{\{B/R_c,\;i\}}$ (rad/s)'); %xlabel('$t$ (s)');
1574 legend('$^B\omega_{\{B/R_c,1\}}$', '$^B\omega_{\{B/R_c,2\}}$', '$^B\omega_{\{B/R_c,3\}}$', '
    Location', 'eastoutside');
1575
1576
1577 nexttile
1578 plot(t_hist, u_vec_hist(1,:), 'LineWidth',1.2); hold on;
1579 plot(t_hist, u_vec_hist(2,:), 'LineWidth',1.2); hold on;
1580 plot(t_hist, u_vec_hist(3,:), 'LineWidth',1.2); hold on;
1581 xlim([t_hist(1) t_hist(end)])
1582 add_mode_shading(t_hist, mode_hist, [-0.005,0.005])
1583 ylim([-0.005,0.005]);
1584 grid on;
1585 ylabel('$^Bu_i$ (kgm$^2s^{-2}$)'); xlabel('$t$ (s)');
1586 legend('$^Bu_1$', '$^Bu_2$', '$^Bu_3$', 'Location', 'eastoutside');
1587
1588
1589
1590 exportgraphics(gcf, 'Mission_2.pdf', 'ContentType','vector');

```